

计算机领域本科教育教学改革试点
工作计划（“101计划”）研究成果

人工智能引论

吴飞

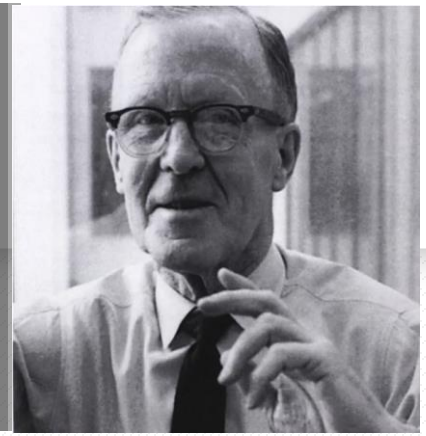
浙江大学计算机学院
浙江大学人工智能研究所

第五章

神经网络与深度学习

Neurons that fire together, wire together

——Donald Hebb



深度学习模型受到了神经系统层次化结构特性的启发，以“层层递进、逐层抽象”机制来完成特定任务。

1949年心理学家唐纳德 赫布提出了有名的赫布理论(Hebbian theory）：神经元之间持续重复经验刺激可导致突触传递效能增加(Neurons that fire together, wire together)，即学习和记忆的本质是大脑结构在后期经验刺激下改变神经元之间强弱联结的权重的结果。

提纲

一、前馈神经网络与参数优化

二、卷积神经网络

三、循环神经网络

四、注意力机制与正则化

五、自然语言和计算机视觉应用

人工神经网络概述

1943 年，神经科学家瓦伦·麦卡洛克（Warren McCulloch）和逻辑学家沃尔特·皮茨（Walter Pitts）合作提出了“**McCulloch–Pitts（MCP）neuron**”的思想。

历史意义：我们在科学史上第一次知道了我们是怎么知道的（for the first time in the history of science, we know how we know)”

神经网络基本单元
MCP神经元

1949 年，心理学家唐纳德·赫布（Donald Hebb）提出了赫布理论(Hebbian theory）：神经元之间持续**重复经验刺激**可导致突触传递效能增加(Neurons that fire together, wire together)”

历史意义：神经元之间突触的强弱变化是学习与记忆的生理学基础

神经元因何链接
赫布理论

20世纪50年代弗朗克·罗森布拉特（Frank Rosenblatt）提出“**感知机**（perceptron）”模型，其是一个仅包含输入层和输出层的两层神经网络，可完成两类问题的线性分类。

历史意义：感知机模型被誉为 “Electronic 'Brain' Teaches Itself（能自学的电脑）”

神经元链接成“网”
感知机

人工神经网络概述

1974 年Werbos在博士论文中提出**误差后向传播**算法，后由鲁梅尔哈特（Rumelhart）和欣顿（Hinton）等人（Rumelhart etal.，1986）完善，解决了多层感知机中参数优化这一难题。

历史意义：以数据驱动方式来根据输出误差自动优化神经网络参数成为可能

神经元之间刺激可层层递进学习
误差后向传播

2006年，欣顿在Science等期刊上发表论文，首次提出“深度信念网络（deep belief network）”模型（Hinton etal.，2006），在相关分类任务中可取得性能超过传统浅层学习模型（如支持向量机）的结果。

历史意义：有别于分段式学习，端到端(end to end)的深度学习框架建立

逐层抽象、层层递进
深度学习

- RNN、LSTM
- CNN
- Word2vec
- BERT、Transformer、LLM

历史意义：从造人和造脑到更通用人工智能

引发AI第三次崛起
语言大模型

前馈神经网络：神经元

- 在生物学中，神经元细胞有**兴奋与抑制**两种状态。大多数神经元细胞在正常情况下处于抑制状态，一旦某个神经元受到刺激并且电位超过一定的阈值后，这个神经元细胞就被激活，处于兴奋状态，并向其他神经元传递信息。
- 基于神经元细胞的结构特性与信息传递方式，神经科学家瓦伦·麦卡洛克和逻辑学家沃尔特·皮茨合作提出了MCP模型。在人工神经网络中，MCP模型成为最基本的结构
- 从图5.1可见，给定n个二值化（0或1）的输入数据$x_i(1 \le i \le n)$与连接参数$w_i(1 \le i \le n)$，MCP神经元模型对输入数据线性加权求和，然后使用函数$\boldsymbol{\Phi}(\cdot)$将加权累加结果映射为0或1，以完成两类分类任务。

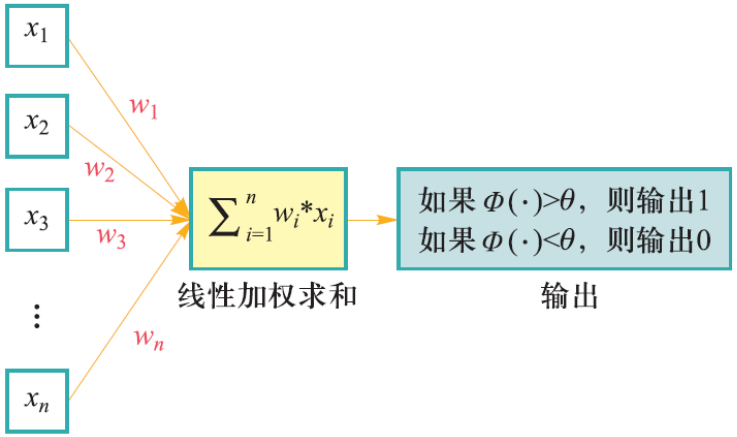


图 5.1　MCP 模型结构

$$y = \boldsymbol{\Phi}\left(\sum_{i=1}^{n} w_i x_i\right)$$

其中w_i为预先设定的连接权重值（一般在0和1中一个值或者1和-1中取一个值），用来表示其所对应输入数据对输出结果的影响（即权重）。$\Phi(\cdot)$将输入端数据与连接权重所得线性加权累加结果与预先设定阈值θ进行比较，根据比较结果输出1或0

前馈神经网络：神经元

在科学史上第一次，我们知道了我们是怎么知道的
(For the first time in the history of science we know how we know and hence are able to state it clearly)

- 从另外一个角度来看，对于任何输入数据$x_i(1 \le i \le n)$，MCP模型可得到1或0这样的输出结果，也就是实现了将输入数据分类到1或0两个类别中，解决了二分类问题。
- Warren McCulloch于1954年在《英国科学哲学期刊》发表的一篇题为《走出形而上学的洞穴》（Through the Den of the Metaphysician）文章中激动骄傲写道，“在科学史上第一次，我们知道了我们是怎么知道的（For the first time in the history of science we know how we know and hence are able to state it clearly）”，认为可通过如上计算方式来模拟神经元对输入信号进行抑制和兴奋的机制。若干个MCP神经元前后相连就构成了最早神经网络雏形。MCP神经元模型虽简单，却开启了建立神经网络大厦地基的征途。

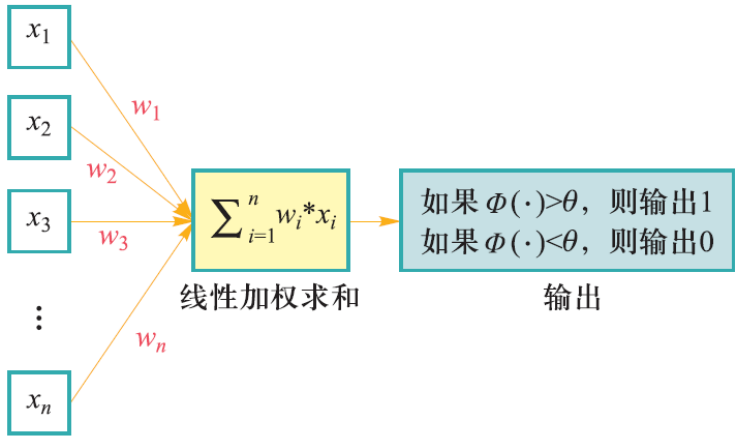


图 5.1　MCP 模型结构

$$y = \Phi\left(\sum_{i=1}^{n} w_i x_i\right)$$

前馈神经网络：感知机

- 早期的感知机结构和MCP模型相似，由一个输入层和一个输出层构成，因此也被称为“单层感知机”。感知机的输入层负责接收实数值的输入向量，输出层则能输出1或-1两个值。单层感知机可作为一种两类线性分类模型，如图5.2所示。

- 对于具有n个输入以及对应连接权重系数为w_i的感知机，首先通过线性加权得到输入数据的累加结果z：$z = w_1x_1 + w_2x_2 + \cdots + b$。这里$x_1, x_2 \ldots x_n$为感知机的输入，b为偏置项（bias）。然后将z作为激活函数$\boldsymbol{\phi}(\cdot)$的输入，激活函数会将z与某一阈值（此例中，阈值为0）进行比较，如果大于该阈值则感知器输出为1，否则输出为-1。通过这样的操作，输入数据被分类为1或-1这两个不同类别。

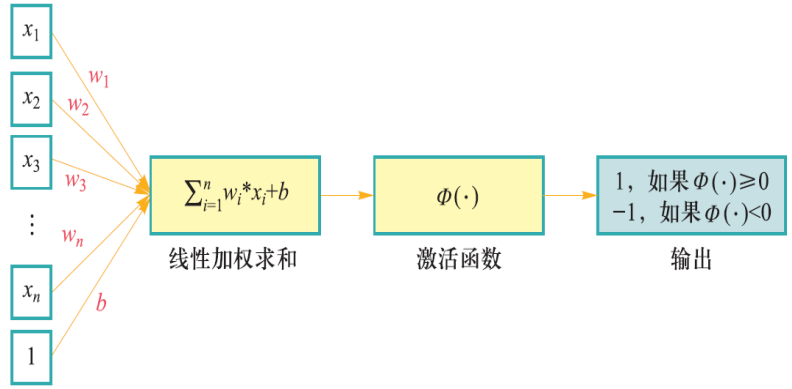


图 5.2　感知机模型

- 感知机中的连接权重参数并不是预先设定好的，而是通过多次迭代训练得到的。具体而言，单层感知机构建损失函数，来计算模型预测值与数据真实值之间的“误差”，通过最小化损失函数取值，来优化模型参数。

前馈神经网络：冰火两重天的感知机

Electronic 'Brain' Teaches Itself

The Navy last week demonstrated the embryo of an electronic computer named the Perceptron which, when completed in about a year, is expected to be the first non-living mechanism able to "perceive, recognize and identify its surroundings without human training or control." Navy officers demonstrating a preliminary form of the device in Washington said they hesitated to call it a machine because it is so much like a "human being without life."

Dr. Frank Rosenblatt, research psychologist at the Cornell Aeronautical Laboratory, Inc., Buffalo, N. Y., designer of the Perceptron, conducted the demonstration. The machine, he said, would be the first electronic device to think as the human brain. Like humans, Perceptron will make mistakes at first, "but it will grow wiser as it gains experience," he said.

recognize the difference between right and left, almost the way a child learns.

When fully developed, the Perceptron will be designed to remember images and information it has perceived itself, whereas ordinary computers remember only what is fed into them on punch cards or magnetic tape.

Later Perceptrons, Dr. Rosenblatt said, will be able to recognize people and call out their names. Printed pages, longhand letters and even speech commands are within its reach. Only one more step of development, a difficult step, he said, is needed for the device to hear speech in one language and instantly translate it to speech or writing in another language.

Self-Reproduction

In principle, Dr. Rosenblatt said, it would be possible to build Per-

1958年7月13日罗森布拉特所写文章

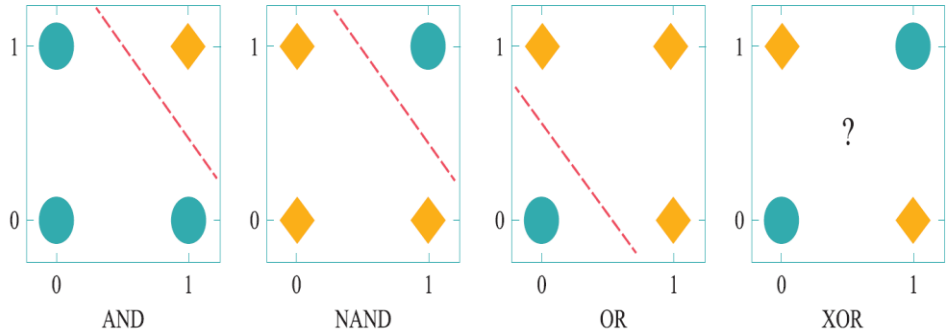


图 5.3　单层感知机模拟不同逻辑函数功能示意

感知机可模拟逻辑与(AND)、逻辑与非(NAND)和逻辑或(OR)等线性可分函数，但是无法完成逻辑异或（XOR）这一非线性可分逻辑函数任务

前馈神经网络

- 在感知机模型中增加若干隐藏层，增强神经网络的非线性表达能力，就会让神经网络具有更强的拟合能力。因此，由多个隐藏层构成的多层感知机被提出，这也被称为前馈神经网络。
- 如图5.4所示，前馈神经网络由输入层、输出层和若干隐藏层构成。网络中各个隐藏层中神经元可接收相邻前序隐藏层中所有神经元传递而来的信息，经过加工处理后将信息输出给相邻后续隐藏层中所有神经元。
- 前馈神经网络中相邻层所包含的神经元之间通常使用“全连接”方式进行连接。所谓“全连接”是指两个相邻层之间的神经元相互成对连接，但同一层内神经元之间没有连接。前馈神经网络可以模拟复杂非线性函数功能，所模拟函数的复杂性取决于网络隐藏层数目和各层中神经元数目。

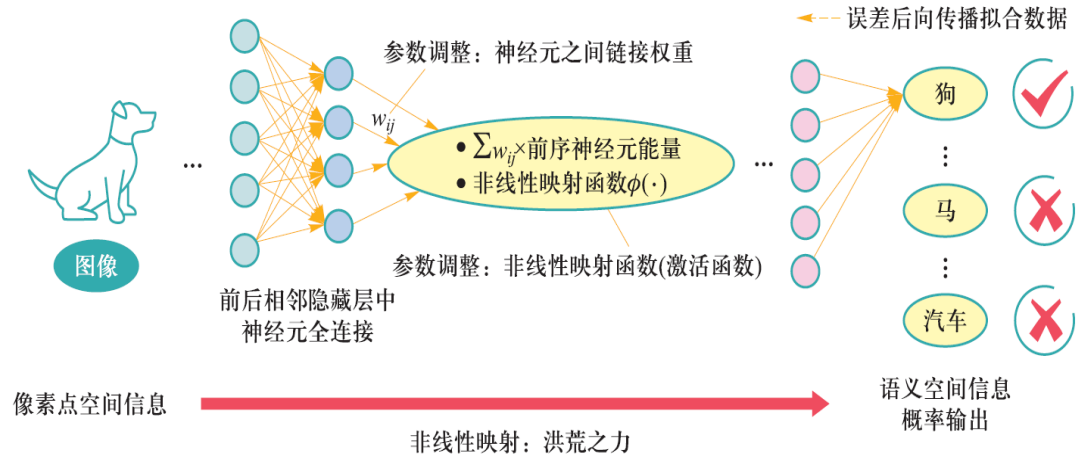


图 5.4　前馈神经网络模型

前馈神经网络

- **层层递进、逐层抽象**：明暗幅度的像素点被映射为高层语义对象的概率值。
- **非线性映射**：恩利克·费米（Enrico Fermi）曾经说过，“圣经中并没有说过一切大自然的定律都可以用线性方程来表示”，亿万个神经元所组成的神经网络就可实现将输入数据将神经网络用于分类任务，将输入数据（如图像、文本和听觉等信息）从一个空间（如像素点空间）映射到另外一个空间（如文本单词空间等），这一过程就是一个复杂的非线性变换。
- **误差反馈调优**：误差后向传播算法会将误差（由损失函数定义）由输出端由后向前传播，逐层去更新神经元链接权重和非线性映射函数等参数，使得神经网络模型拟合数据（data fitting），让神经元之间记住这种刺激或者强化信息流通路，打通“任督二脉”，从而在输入数据和输出结果之间建立关联。

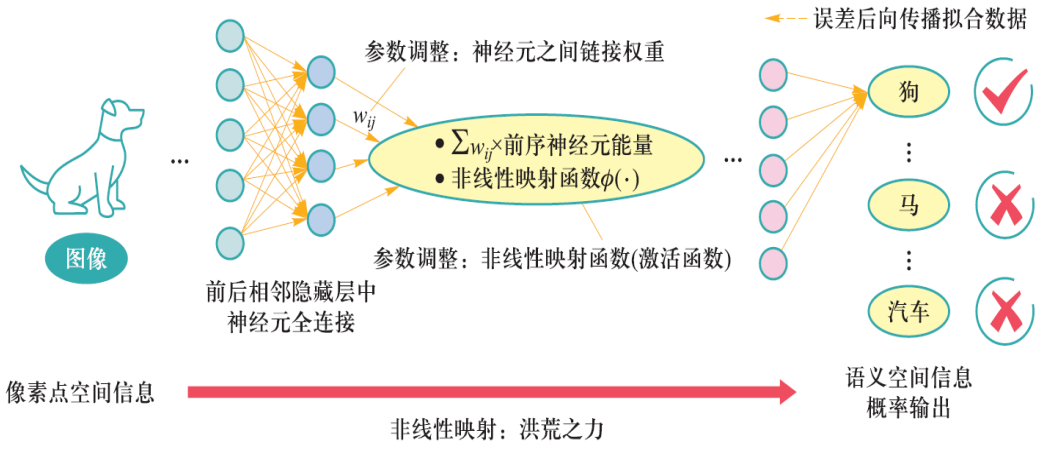


图 5.4　前馈神经网络模型

前馈神经网络：执行非线性映射的激活函数

sigmoid函数 $f(x)=\frac{1}{1+e^{-x}}$

优点：1）概率形式输出。sigmoid函数值域为(0，1)，因此Sigmoid函数的输出可视为概率值；2）单调递增。Sigmoid函数对输入x取值范围没有限制，但当x大于一定值后，函数输出无限趋近于1，而小于一定数值后，函数输出无限趋近于0。特别地，当x=0 时，函数输出为0.5；3）非线性变化。x取值在0附近时，函数输出值的变化幅度比较大（函数值变化陡峭），意味着函数在0附近容易被激活且是非线性变化。当x取值很大或很小时，函数输出值几乎不变，这是基于概率的一种认识与需要。

Sigmoid函数的导数可以通过如下方式推导得到：

$$f'(x)=(\frac{1}{1+e^{-x}})'=\frac{e^{-x}}{(1+e^{-x})^2}=\frac{1+e^{-x}-1}{(1+e^{-x})^2}=\frac{1}{(1+e^{-x})}\left(1-\frac{1}{(1+e^{-x})}\right)=f(x)(1-f(x))$$

由于$0<f(x)<1$和$0<1-f(x)<1$，因此Sigmoid函数的导数$f(x)(1-f(x))$小于1，在使用反向传播算法更新参数过程中易出现导数过于接近于0的情况，即梯度消失（vanishing gradient）的问题，并且这个问题会随着网络的深度增加而变得愈发严重。

前馈神经网络：执行非线性映射的激活函数

ReLU函数 $f(x)=\begin{cases}0, for\ x<0\\ x, for\ x\geq 0\end{cases}$ 或者$f(x)=\max(0,x)$

ReLU函数的导数为：$f'(x)=\begin{cases}0, for\ x<0\\ 1, for\ x\geq 0\end{cases}$

当输入$x\geq 0$时，ReLU的导数为常数，这样可有效缓解梯度消失这一问题。当$x<0$时， ReLU的梯度总是0，导致神经网络中若干参数激活值为0，即参与分类等任务神经元数目稀缺，这种稀疏性可以在一定程度上克服机器学习中经常出现的过拟合现象。然而，当 x < 0 时，ReLU梯度为0也导致神经元“死亡”，即神经元对应权重永远不会更新。为了解决这个问题，可以使用其他激活函数，如Leaky ReLU 或Parametric ReLU，它们在x < 0时梯度取值非零。

前馈神经网络：执行非线性映射的激活函数

Softmax激活函数 $y_i = \text{softmax}(x_i) = \dfrac{e^{x_i}}{\sum_{j=1}^{k} e^{x_j}}$

Softmax函数一般用于多分类问题中，它是对逻辑斯蒂（logistic）回归的一种推广，也被称为多项逻辑斯蒂回归模型(multi-nominal logistic mode)。假设要实现k个类别的分类任务，Softmax函数将输入数据x_i映射到第i个类别的概率y_i如下计算:

显然，$0 < y_i < 1$。图5.5给出了三类分类问题的Softmax输出示意图。在图中，对于取值为4、1和-4的x_1、x_2和x_3，通过softmax变换后，将其映射到(0,1)之间的概率值。

由于Softmax输出结果的值累加起来为1，因此可将输出概率最大的作为分类目标（图5.5中被分类为第一类）。

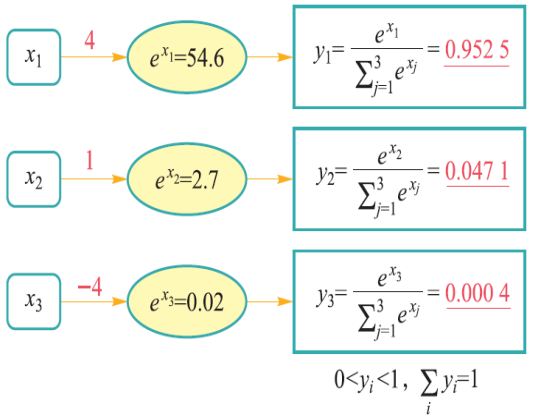


图 5.5　三分类问题的 softmax 输出示意

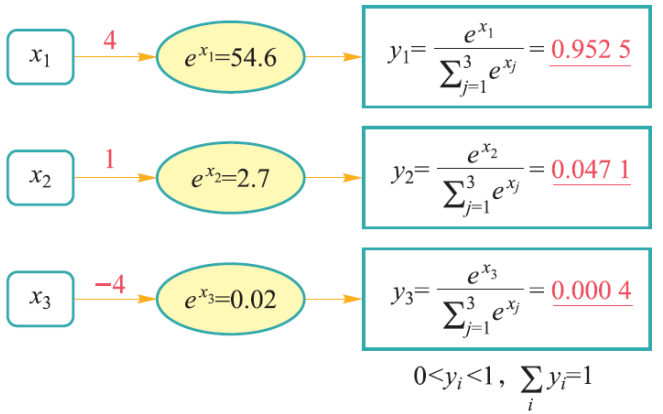


图 5.5　三分类问题的 softmax 输出示意

前馈神经网络：神经网络参数优化

- 在设计完成一个多层神经网络的结构后（比如多少个隐藏层，每个隐藏层中包含的神经元数目以及神经元采用的非线性激活函数等），需要对神经网络中参数（如神经元之间连接权重系数以及激活函数系数等）进行优化学习，以使得神经网络能够将给定输入数据映射到所期望的输出语义空间（如将一幅像素点构成的图像数据映射为人脸），完成分类识别等任务。

- 神经网络参数优化是一个监督学习的过程。给定n个标注样本数据(x_i, y_i)，这里x_i是输入数据、y_i是x_i的类别等标注信息。假设输入数据x_i经过神经网络处理后所得输出预测值为$\hat{y}_i$，则可通过损失函数来计算模型预测值$\hat{y}_i$与真实值y_i之间误差或损失，利用其来优化模型参数，以便让神经网络按照x_i的真实信息y_i输出，即让神经网络对数据进行拟合（data fitting）。

- 具体而言，模型会利用反向传播算法将损失函数计算所得误差从输出端出发、由后向前传递给神经网络中每个单元，然后通过梯度下降算法对神经网络中参数进行更新。随着迭代的进行，模型预测所得$\hat{y}_i$与真实值y_i之间的误差逐步缩小，模型预测准确率得以提升，预测的能力得以增强，当迭代达到一定轮次或准确率达到一定水平时，则可认为模型参数已被优化完毕。

神经网络参数优化：损失函数

- 损失函数（loss function）又称为代价函数（cost function），用来计算模型预测值与真实值之间的误差。损失函数是神经网络设计中的一个重要组成部分。通过定义与任务相关的良好损失函数，在训练过程中可根据损失函数来计算神经网络的误差大小，进而优化神经网络参数。
- 两种最常用的损失函数：均方误差损失函数和交叉熵损失函数。

均方误差损失函数

$$\mathrm{MSE}=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$$

均方误差损失函数通过计算预测值和实际值之间距离（即误差）的平方来衡量模型优劣。假设有n个训练数据x_i，每个训练数据x_i的真实输出为y_i，模型对x_i的预测值为$\hat{y}_i$。该模型在n个训练数据下所产生的均方误差损失可定义如上。

神经网络参数优化：损失函数

交叉熵损失函数 $\mathrm{H}(y_i, \hat{y}_i) = -y_i * log\hat{y}_i$

- “熵”被用来表示热力学系统所呈现的无序程度。香农将这一概念引入信息论领域，提出了“信息熵”概念，通过对数函数来测量信息的不确定性。
- 交叉熵（cross entropy）是信息论中的重要概念，主要用来度量两个概率分布间的差异。y_i是样本x_i分类的真实概率、$\hat{y}_i$是模型预测概率，通过$\hat{y}_i$来表示y_i的交叉熵可如上计算。
- 交叉熵刻画了两个概率分布之间的距离，旨在描绘通过概率分布$\hat{y}_i$来表达概率分布y_i的困难程度。根据公式不难理解，两个概率分布y_i和$\hat{y}_i$越接近，交叉熵越小。也就是说，预测类别分布概率与实际类别分布概率之间的差距越小，交叉熵就越小。

这里以一个示例子来说明这个事实。假设模型 1 和模型 2 分别对三个分别属于类别 1、类别 2 和类别 3 的样本进行分类，得到如表 5.2 所示的分类结果。

表 5.2　两个模型的分类结果

样本	真实值	模型 1 预测结果	模型 2 预测结果
样本 1（类别 1）	0 0 1	0.3 0.3 0.4	0.1 0.2 0.7
样本 2（类别 2）	0 1 0	0.3 0.4 0.3	0.1 0.7 0.2
样本 3（类别 3）	1 0 0	0.1 0.2 0.7	0.3 0.4 0.3

模型 1 在三个样本上的交叉熵损失平均值为

$$\frac{-(0\times\log 0.3+0\times\log 0.3+1\times\log 0.4+0\times\log 0.3+1\times\log 0.4+0\times\log 0.3+1\times\log 0.1+0\times\log 0.2+0\times\log 0.7)}{3}$$

$$=1.37$$

模型 2 在三个样本上的交叉熵损失平均值为

$$\frac{-(0\times\log 0.1+0\times\log 0.2+1\times\log 0.7+0\times\log 0.1+1\times\log 0.7+0\times\log 0.2+1\times\log 0.3+0\times\log 0.4+0\times\log 0.3)}{3}$$

$$=0.63$$

可见，交叉熵损失函数准确给出了模型 2 预测性能优于模型 1 预测性能的事实。如果模型预测值与真实值一致，则交叉熵损失为零。

神经网络参数优化：梯度下降

梯度下降（gradient descent）算法是机器学习中对模型参数进行优化时所采用的常用方法。为了让模型预测结果与真实结果之间的损失误差最小，利用梯度下降法来逐步迭代优化模型参数。当损失误差最小时，此时学习得到的模型参数即为模型的最优参数。梯度下降**是一种使损失函数最小化的方法。一元变量所构成函数f在x处梯度为：**

$$\frac{df(x)}{dx}=\lim_{h\to 0}\frac{f(x+h)-f(x)}{h}$$



- 在多元函数中，梯度是对每一变量所求导数组成的向量
- 梯度的反方向是函数值下降最快的方向，因此是损失函数求解的方向

神经网络参数优化：梯度下降

假设损失函数$f(\boldsymbol{x})$是连续可微的多元变量函数，其泰勒展开如下：

$$f(\boldsymbol{x}+\Delta\boldsymbol{x})=f(\boldsymbol{x})+f'(\boldsymbol{x})\Delta\boldsymbol{x}+\frac{1}{2}f''(\boldsymbol{x})(\Delta\boldsymbol{x})^2+\cdots+\frac{1}{n!}f^{(n)}(\boldsymbol{x})(\Delta\boldsymbol{x})^n$$

$$f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})\approx(\nabla f(\boldsymbol{x}))^T\Delta\boldsymbol{x}$$

参数优化后　参数优化前　误差减少量

假设参数变量当前取值为$\boldsymbol{x}$，其对应的损失误差数值为$f(\boldsymbol{x})$。在不断促使损失误差减少的下一步迭代时，$\boldsymbol{x}$取值变化为$\boldsymbol{x}+\Delta\boldsymbol{x}$，则损失误差数值相应地变为$f(\boldsymbol{x}+\Delta\boldsymbol{x})$。

由于$f(\boldsymbol{x})$是损失函数，因此在迭代中希望$f(\boldsymbol{x}+\Delta\boldsymbol{x})$取值小于$f(\boldsymbol{x})$，即损失误差随着模型参数被优化而逐渐减少，使得$f(\boldsymbol{x}+\Delta\boldsymbol{x})<f(\boldsymbol{x})$，于是$(\nabla f(\boldsymbol{x}))^T\Delta\boldsymbol{x}<0$。

神经网络参数优化：梯度下降

$$f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})=\|\nabla f(\boldsymbol{x})\|\|\Delta\boldsymbol{x}\|\cos\theta=-\|\Delta\boldsymbol{x}\|\|\nabla f(\boldsymbol{x})\|$$

参数优化后　参数优化前　　误差减少量

- 在$(\nabla f(\boldsymbol{x}))^T\Delta\boldsymbol{x}=\|\nabla f(\boldsymbol{x})\|\|\Delta\boldsymbol{x}\|\cos\theta$中，$\|\nabla f(\boldsymbol{x})\|$和$\|\Delta\boldsymbol{x}\|$分别为损失函数梯度的模和下一轮迭代中$\boldsymbol{x}$取值增量的模，两者均为正数。为了保证损失误差减少，只要保证$\cos\theta<0$。当$\theta=180°$时，$\cos\theta=-1$，这时损失函数减少的幅度值$(\nabla f(\boldsymbol{x}))^T\Delta\boldsymbol{x}$取到最小。可见，当损失函数梯度$\nabla f(\boldsymbol{x})$和$\boldsymbol{x}$取值增量$\Delta\boldsymbol{x}$之间夹角为180度时，损失误差减少最多。于是，在迭代过程中为了最小化损失误差，每次迭代时可沿着损失函数梯度的反方向（即$\theta=180°$）以$\Delta\boldsymbol{x}$幅度来改变$\boldsymbol{x}$取值，这样会保证$f(\boldsymbol{x}+\Delta\boldsymbol{x})$和$f(\boldsymbol{x})$之间的差值最大（即损失误差下降最快）。
- 于是，当$\theta=180°$时，$f(\boldsymbol{x}+\Delta\boldsymbol{x})$和$f(\boldsymbol{x})$之间的差值为：$f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})=\|\nabla f(\boldsymbol{x})\|\|\Delta\boldsymbol{x}\|\cos\theta=-\|\Delta\boldsymbol{x}\|\|\nabla f(\boldsymbol{x})\|$。这说明只要沿着损失函数梯度的反方向选取$\boldsymbol{x}$的增量$\Delta\boldsymbol{x}$，就会保证损失误差下降最多、下降最快，犹如从山峰处沿着最陡峭路径可快速走到山谷。在前面的推导中忽略了损失函数的二阶导数以及其高阶导数的取值，因此在实际中引入步长η，用$\boldsymbol{x}-\eta\nabla f(\boldsymbol{x})$来更新$\boldsymbol{x}$（在具体实现时$\eta$可取一个定值）。

神经网络参数优化：误差反向传播

- 误差反向传播算法（error back propagation）主要利用损失函数来计算模型预测结果与真实结果之间误差以优化调整模型参数，这一优化调整机制是从输出端向输入端，由后向前递进进行。
- 损失函数梯度反方向是损失误差下降最快方向。在求取损失函数梯度时需要对各个变量求偏导，这一求取偏导过程要用到链式法则（chain rule）。
- 在图5.6中，out是一个神经单元，其在神经网络中完成两个任务：1）将相邻前序神经元传递而来的信息线性加权累加，得到结果$\mathcal{X}$，即$\mathcal{X} = \sum_i w_i \times out_i (1 \le i \le 3)$。这里$out_i$和$w_i$分别是与$out$前序相连的第$i$个神经元后向输递信息及第$i$个神经元与$out$之间的连接权重系数；2）使用激活函数对加权累加结果进行非线性变换。图中使用的激活函数为sigmoid，即$\mathcal{O} = \frac{1}{1+e^{-\mathcal{X}}}$。可见，$\mathcal{O}$的大小就是神经网络对输入样本数据的预测输出。

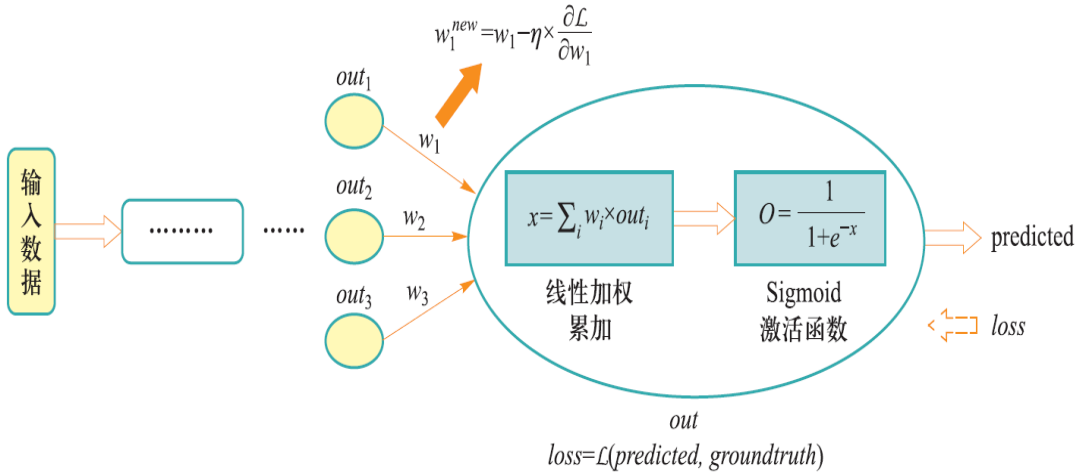


图 5.6　链式求导与模型参数更新示意

神经网络参数优化：误差反向传播

模型预测结果O与真实结果之间的损失误差$loss$由所定义的损失函数$\mathcal{L}$计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数w_i（未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

如何调整w_1

为了使损失函数$\mathcal{L}$取值减少（从而保证模型预测结果与实际结果之间的差距越来越小），需要求取损失函数$\mathcal{L}$相对于w_1的偏导，然后按照损失函数梯度的反方向选取一个微小的增量，来调整w_1的取值，就能够保证损失函数取值减少。即将w_1变为$w_1 - \eta\frac{d\mathcal{L}}{dw_1}$后，能使得损失误差减少。这里$\frac{d\mathcal{L}}{dw_1}$为损失函数$\mathcal{L}$对$w_1$的偏导。于是，需要求取损失函数$\mathcal{L}$对变量参数$w_1$的偏导。

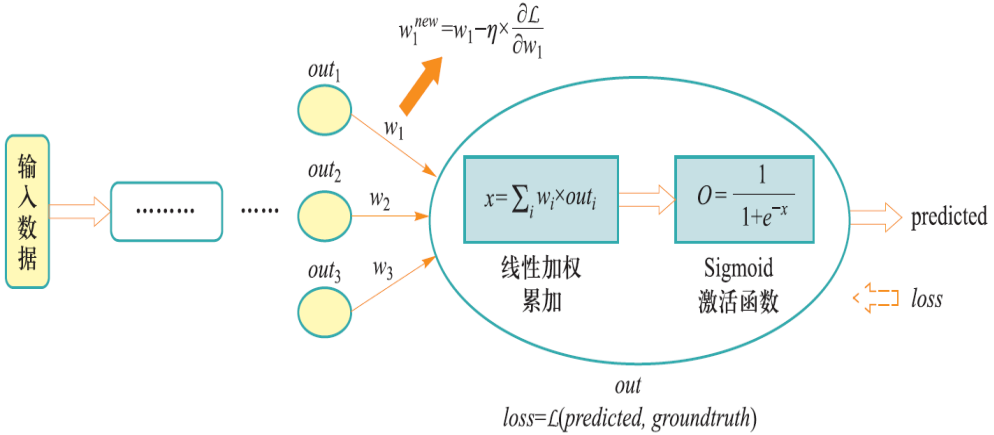


图 5.6　链式求导与模型参数更新示意

神经网络参数优化：误差反向传播

模型预测结果O与真实结果之间的损失误差$loss$由所定义的损失函数$\mathcal{L}$计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数w_i（未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

如何调整w_1

由于w_1与加权累加函数$\mathcal{X}$和sigmoid函数均有关，因此$\frac{d\mathcal{L}}{dw_1}=\frac{d\mathcal{L}}{dO}\frac{dO}{d\mathcal{X}}\frac{d\mathcal{X}}{dw_1}$。在这个链式求导中：$\frac{d\mathcal{L}}{dO}$与损失函数的定义有关；$\frac{dO}{d\mathcal{X}}$是对sigmoid函数求导，结果为$\frac{1}{1+e^{-x}}\times(1-\frac{1}{1+e^{-x}})$；$\frac{d\mathcal{X}}{dw_1}$是加权累加函数$w_1\times out_1+w_2\times out_2+w_3\times out_3$对$w_1$求导，结果为$out_1$。

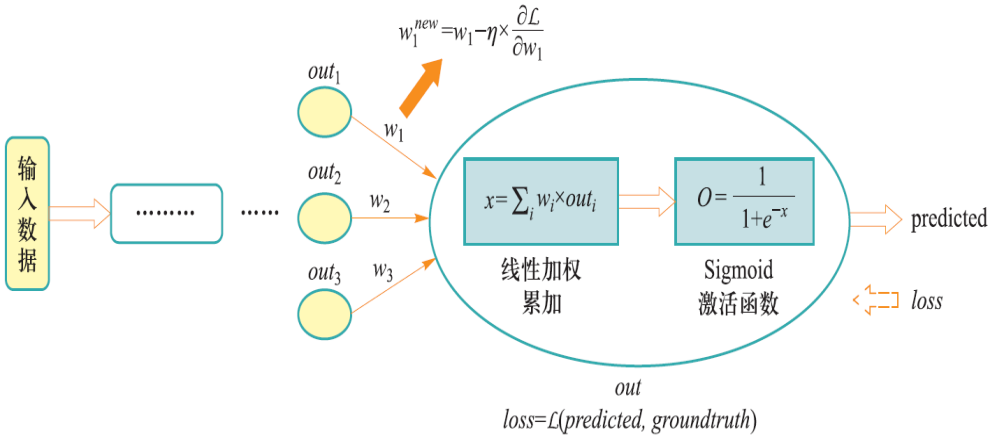


图 5.6　链式求导与模型参数更新示意

神经网络参数优化：误差反向传播

模型预测结果O与真实结果之间的损失误差$loss$由所定义的损失函数$\mathcal{L}$计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数w_i（未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

链式求导

- 链式求导实现了损失函数对某个自变量求偏导，好比将损失误差从输出端向输入端逐层传播，通过这个传播过程来更新该自变量取值。梯度下降法告诉我们，只要沿着损失函数梯度的反方向来更新参数，就可使得损失函数下降最快。
- 因此，假设损失函数$\mathcal{L}(\theta)$（θ为参数），可使用如下梯度下降算法对参数θ进行更新：$\theta^{new} = \theta - \eta \times \nabla_\theta \mathcal{L}(\theta)$。其中$\eta$也被称学习速率（learning rate），其大小会影响模型参数收敛速度，$\nabla_\theta \mathcal{L}(\theta)$是损失函数对参数$\theta$求偏导结果。神经网络中任意一个参数相对于损失函数的导数都可通过链式求导得到（这一求导过程可视为误差后向传播过程）。

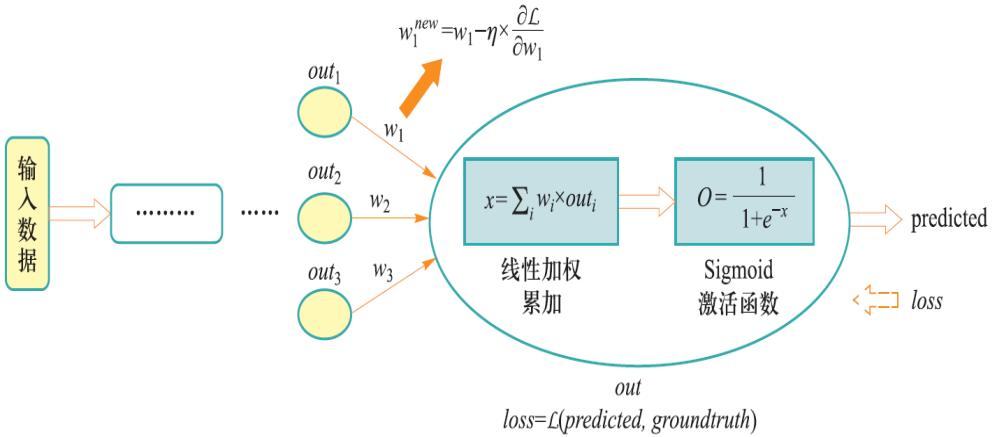


图 5.6　链式求导与模型参数更新示意

神经网络参数优化：误差反向传播

模型预测结果O与真实结果之间的损失误差$loss$由所定义的损失函数$\mathcal{L}$计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数w_i（未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

通过链式求导调整调整w_1

- 在图5.6中，参数w_1在下一轮迭代中的取值被调整为：$w_1^{new} = w_1 - \eta \times \frac{d\mathcal{L}}{dw_1} = w_1 - \eta \times \frac{d\mathcal{L}}{dO}\frac{dO}{dX}\frac{dX}{dw_1} = w_1 - \eta \times \frac{d\mathcal{L}}{dO} \times \frac{1}{1+e^{-X}} \times (1 - \frac{1}{1+e^{-X}}) \times out_1$。
- 按照同样的方法，可调整w_2、w_3以及其它图中未显示参数的取值。经过这样的调整，在模型参数新的取值作用下，损失函数$\mathcal{L}(\theta)$会以最快下降方式逐渐减少，直至减少到最小值（即全局最小值）。

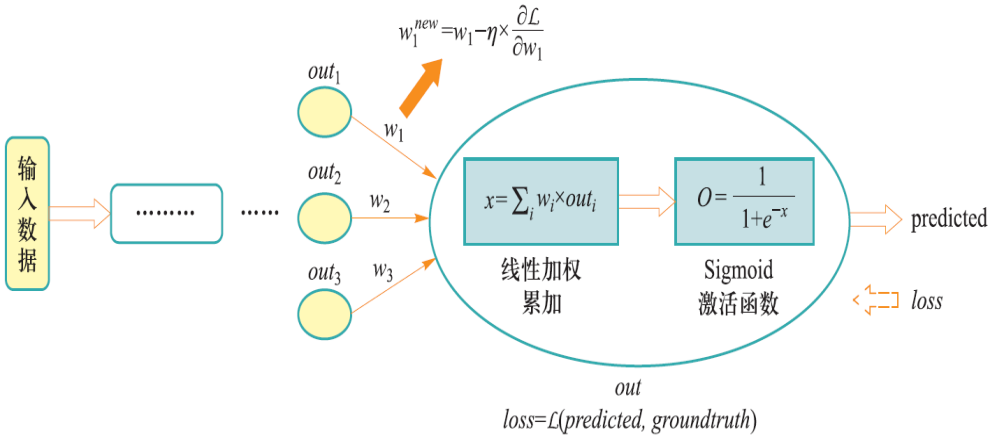


图 5.6　链式求导与模型参数更新示意

神经网络参数优化：梯度下降的方法

梯度下降算法可以分为**批量梯度下降算法**（batch gradient descent）、**随机梯度下降算法**（stochastic gradient descent）和**小批量梯度下降算法**（mini-batch gradient descent）等类型。

- **批量梯度下降：在整个训练集上计算损失误差$\mathcal{L}(\theta)$。如果数据集较大，则会因内存容量不足而无法完成梯度计算。**
- **随机梯度下降：使用训练集中每个训练样本计算$\mathcal{L}(\theta)$来分别更新参数。因为随机梯度下降每次迭代只使用一个样本，导致梯度方向具有很大的波动，因此其收敛速度通常较慢。尽管如此，随机梯度下降的随机性有助于跳出局部最优解，有时可能找到更好的解。**
- **小批量梯度下降：选取训练集中小批量样本计算$\mathcal{L}(\theta)$，根据每一批量样本所得到的累加误差来更新参数，这样可保证训练过程更稳定，而且采用批量训练方法也可利用矩阵计算优势，因此其成为目前最常用的梯度下降算法**

神经网络参数优化：误差反向传播与参数更新例子

- 下面通过一个三类分类的具体例子来介绍神经网络中参数更新过程。给定一个包含输入层、一层隐藏层和输出层的多层感知机（如图5.7所示），其中隐藏层由两个神经元构成。这里为了简化介绍过程，因此只使用了一层隐藏层。网络使用Sigmoid函数作为神经元的激活函数，使用均方损失函数来计算网络输出值与实际值之间的误差。

- 在图5.7中，每个神经元完成如下两项任务：1）对相邻前序层所传递信息进行线性加权累加；2）对加权累加结果进行非线性变换。假设样本数据输入为(x_1, x_2, x_3)，其标注信息为(y_1, y_2, y_3)。在三类分类问题中，对于输入数据(x_1, x_2, x_3)，y_1、y_2和y_3中只有一个取值为1、其余两个取值为0。

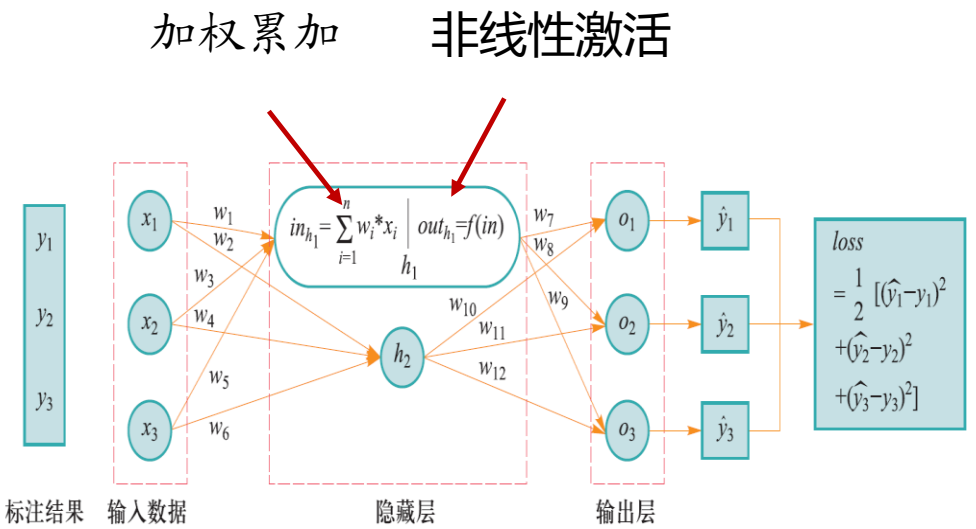


图 5.7　包含一个隐藏层的前馈神经网络

神经网络参数优化：误差反向传播与参数更新例子

表 5.3　图 5.7 中神经网络各单元的计算结果

单元	相邻前序神经元所传递信息的线性累加	非线性变换
h_1	$\text{In}_{h_1} = w_1 \times x_1 + w_3 \times x_2 + w_5 \times x_3$	$\text{Out}_{h_1} = \text{sigmoid}(\text{In}_{h_1})$
h_2	$\text{In}_{h_2} = w_2 \times x_1 + w_4 \times x_2 + w_6 \times x_3$	$\text{Out}_{h_2} = \text{sigmoid}(\text{In}_{h_2})$
o_1	$\text{In}_{o_1} = w_7 \times \text{Out}_{h_1} + w_{10} \times \text{Out}_{h_2}$	$\hat{y}_1 = \text{Out}_{o_1} = \text{sigmoid}(\text{In}_{o_1})$
o_2	$\text{In}_{o_2} = w_8 \times \text{Out}_{h_1} + w_{11} \times \text{Out}_{h_2}$	$\hat{y}_2 = \text{Out}_{o_2} = \text{sigmoid}(\text{In}_{o_2})$
o_3	$\text{In}_{o_3} = w_9 \times \text{Out}_{h_1} + w_{12} \times \text{Out}_{h_2}$	$\hat{y}_3 = \text{Out}_{o_3} = \text{sigmoid}(\text{In}_{o_3})$

加权累加　　非线性激活

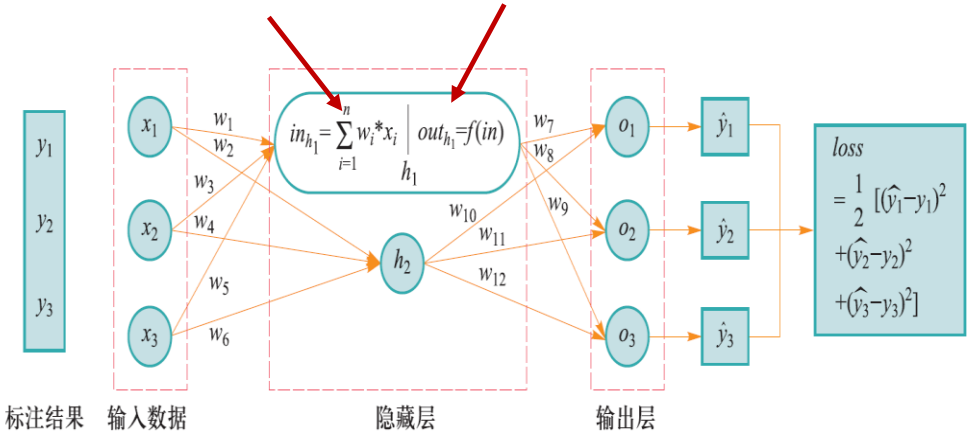


图 5.7　包含一个隐藏层的前馈神经网络

神经网络参数优化：误差反向传播与参数更新例子---计算误差

- 一旦神经网络在当前参数下给出了预测结果 $(\widehat{y_1},\widehat{y_2},\widehat{y_3})$ 后，通过均方误差损失函数来计算模型预测值与真实值 (y_1,y_2,y_3)之间误差，记为$loss=\frac{1}{3}\sum_{i=1}^{3}(\hat{y}_i-y_i)^2$。
- 接下来通过梯度下降和误差反向传播方法，沿着损失函数梯度的反方向来更改参数变量取值，使得损失函数快速下降到其最小值。由于损失函数对w_7~w_{12}的偏导计算相似，在此以w_7为例来介绍如何更新w_7这一参数取值。记损失函数为$\mathcal{L}(w)$，$\mathcal{L}(w)=\frac{1}{3}\sum_{i=1}^{3}(\hat{y}_i-y_i)^2$。

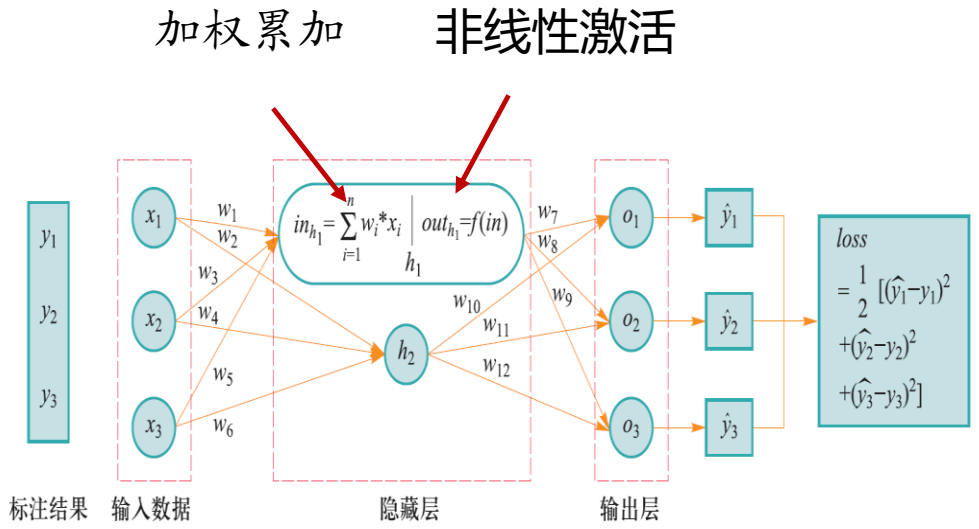


图 5.7　包含一个隐藏层的前馈神经网络

神经网络参数优化：误差反向传播与参数更新例子---计算误差

损失函数$\mathcal{L}$对参数w_7的偏导可如下计算:

$$\delta_7 = \frac{\partial \mathcal{L}}{\partial w_7} = \frac{\partial \mathcal{L}}{\partial \widehat{y_1}} * \frac{\partial \widehat{y_1}}{\partial In_{o_1}} * \frac{\partial In_{o_1}}{\partial w_7}$$

上述公式中每一项结果如下:

$$\frac{\partial \mathcal{L}}{\partial \widehat{y_1}} = \widehat{y_1} - y_1$$

$$\frac{\partial \widehat{y_1}}{\partial In_{o_1}} = \widehat{y_1} * (1 - \widehat{y_1})$$

$$\frac{\partial In_{o_1}}{\partial w_7} = Out_{h_1}$$

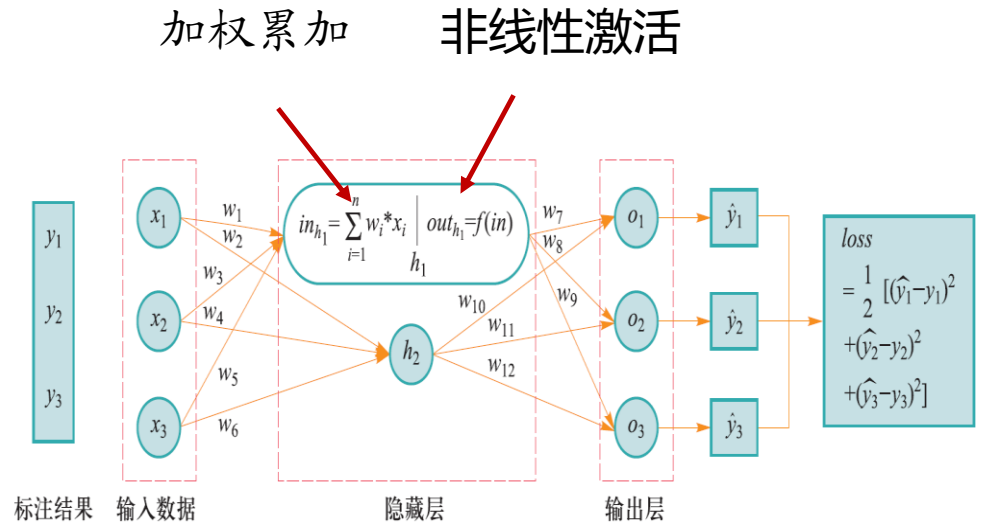


图 5.7　包含一个隐藏层的前馈神经网络

神经网络参数优化：误差反向传播与参数更新例子---计算误差

在得到w_7~w_{12}相对于损失函数的偏导结果后，利用链式求导法则，可计算得到损失函数相对于w_1~w_6的偏导结果。下面以w_1为例介绍损失函数$\mathcal{L}$对w_1的偏导δ_1。

$$\delta_1 = \frac{\partial \mathcal{L}}{\partial w_1}$$
$$= \frac{\partial \mathcal{L}}{\partial \widehat{y_1}} * \frac{\partial \widehat{y_1}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \widehat{y_2}} * \frac{\partial \widehat{y_2}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \widehat{y_3}} * \frac{\partial \widehat{y_3}}{\partial w_1}$$
$$= \frac{\partial \mathcal{L}}{\partial \widehat{y_1}} * \frac{\partial \widehat{y_1}}{\partial In_{o_1}} * \frac{\partial In_{o_1}}{\partial Out_{h_1}} * \frac{\partial Out_{h_1}}{\partial In_{h_1}} * \frac{\partial In_{h_1}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \widehat{y_2}} * \frac{\partial \widehat{y_2}}{\partial In_{o_2}} * \frac{\partial In_{o_2}}{\partial Out_{h_1}} * \frac{\partial Out_{h_1}}{\partial In_{h_1}} * \frac{\partial In_{h_1}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \widehat{y_3}} * \frac{\partial \widehat{y_3}}{\partial In_{o_3}}$$
$$* \frac{\partial In_{o_3}}{\partial Out_{h_1}} * \frac{\partial Out_{h_1}}{\partial In_{h_1}} * \frac{\partial In_{h_1}}{\partial w_1}$$
$$= \left(\frac{\partial \mathcal{L}}{\partial \widehat{y_1}} * \frac{\partial \widehat{y_1}}{\partial In_{o_1}} * \frac{\partial In_{o_1}}{\partial Out_{h_1}} + \frac{\partial \mathcal{L}}{\partial \widehat{y_2}} * \frac{\partial \widehat{y_2}}{\partial In_{o_2}} * \frac{\partial In_{o_2}}{\partial Out_{h_1}} + \frac{\partial \mathcal{L}}{\partial \widehat{y_3}} * \frac{\partial \widehat{y_3}}{\partial In_{o_3}} * \frac{\partial In_{o_3}}{\partial Out_{h_1}} \right) * \frac{\partial Out_{h_1}}{\partial In_{h_1}} * \frac{\partial In_{h_1}}{\partial w_1}$$
$$= (\delta_7 + \delta_8 + \delta_9) * \frac{\partial Out_{h_1}}{\partial In_{h_1}} * \frac{\partial In_{h_1}}{\partial w_1}$$

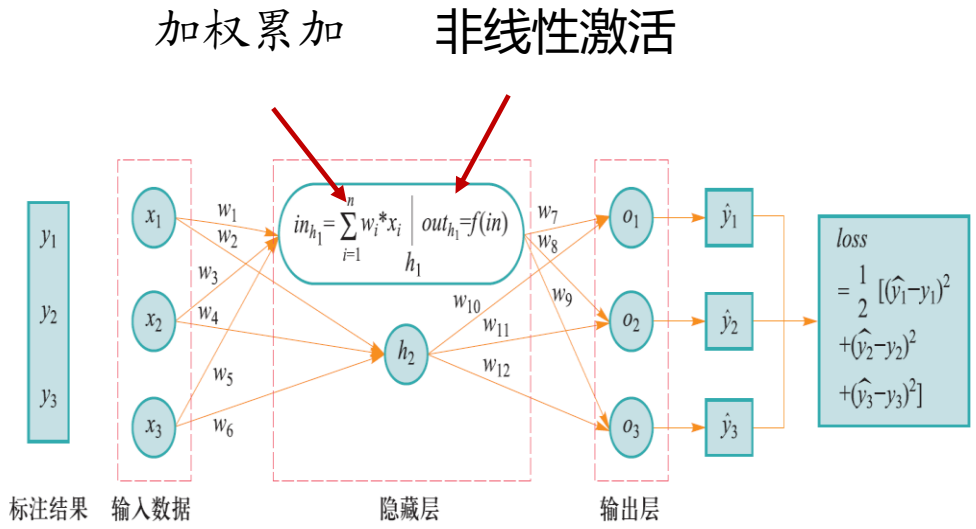


图 5.7　包含一个隐藏层的前馈神经网络

在计算得到所有参数相对于损失函数$\mathcal{L}$的偏导结果后，利用梯度下降算法，通过$w_i^{new} = w_i - \eta * \delta_i$来更新参数取值。然后不断迭代，直至模型参数收敛，此时损失函数减少到其最小值。

卷积神经网络：为什么是它

- 在前馈神经网络中，输入层的输入数据直接与第一个隐藏层中的所有神经元相互连接。如果输入数据是一幅图像，需要把灰度图像（二维矩阵）或彩色图像（三维矩阵）转换为向量形式。例如，给定一幅分辨率为1 000 × 1 000的灰度图像，输入数据卷积神经网络为一个1 000 000维的向量，如果输入数据与第一个隐藏层中的所有神经元均相连，且第一个隐藏层的维度与输入维度一致，则输入层到第一个隐藏层之间待训练的参数数量为1012个，模型参数数量如此巨大，不仅会占用大量计算机内存，而且也会使神经网络模型的训练变得难以收敛。因此，对于图像这样的数据，**不能直接将所构成的像素点向量与前馈神经网络中的每个神经元相连**。

- 1959年，戴维·胡贝尔和托尔斯滕·维泽尔发现了人脑“视觉系统信息分层处理”这一机制，即可视皮层对外界信息是分级感受的。受这一发现启发，1980年，Kunihiko Fukishima对神经科学所发现的结构进行了计算机模拟，提出通过**级联方式（cascade，即逐层滤波）实现一种满足平移不变性的网络Neocognitron，这就是卷积神经网络的前身**（Fukushima，1980，1988）。20世纪90年代，LeCun等人设计了一种被称为LeNet-5的卷积神经网络用于手写体识别，初步确立了卷积神经网络的基本结构。

- 如今，虽然针对不同应用场景的卷积神经网络结构已变得愈发复杂，性能也变得更加强劲，但是究其本质，这些复杂网络结构仍是以卷积操作与池化操作为核心构建而成的。本节对卷积神经网络中的卷积操作、池化操作和网络优化的内容做详细介绍。

卷积神经网络：卷积过程

- 如图5.8所示，给定一个权重分别为$w_i(1 \le i \le 9)$、大小为3×3的卷积核以及一个5×5大小灰度图像。该卷积核对图像的卷积操作就是分别以图像中g、h、i、l、m、n、q、r和s所在像素点位置为中心，形成一个3×3大小的图像子块区域，然后用卷积核所定义权重w_i对图像子块区域内每个像素点进行加权累加，所得结果即为图像子块区域中心像素点被卷积后（即滤波后）的结果。滤波也可视为在给定卷积核权重前提下，记住了邻域像素点之间的若干特定空间模式、忽略了某些空间模式。
- 这里以像素点g为中心像素点进行滤波处理为例，以其为中心所形成的3×3大小子块中包含了其邻域的a、b、c、f、h、k、l、m八个像素点，用卷积核中的权重w_i对这些像素点（含像素点g）进行加权累加，所得到的结果即为像素点g的卷积滤波结果，即$g = w_1 \times a + w_2 \times b + w_3 \times c + w_4 \times f + w_5 \times g + w_6 \times h + w_7 \times k + w_8 \times l + w_9 \times m$。
- 同理可得到像素点h、i、l、m、n、q、r和s的卷积滤波结果（如图5.8），卷积滤波结果在卷积神经网络中被称为特征图（feature map）。

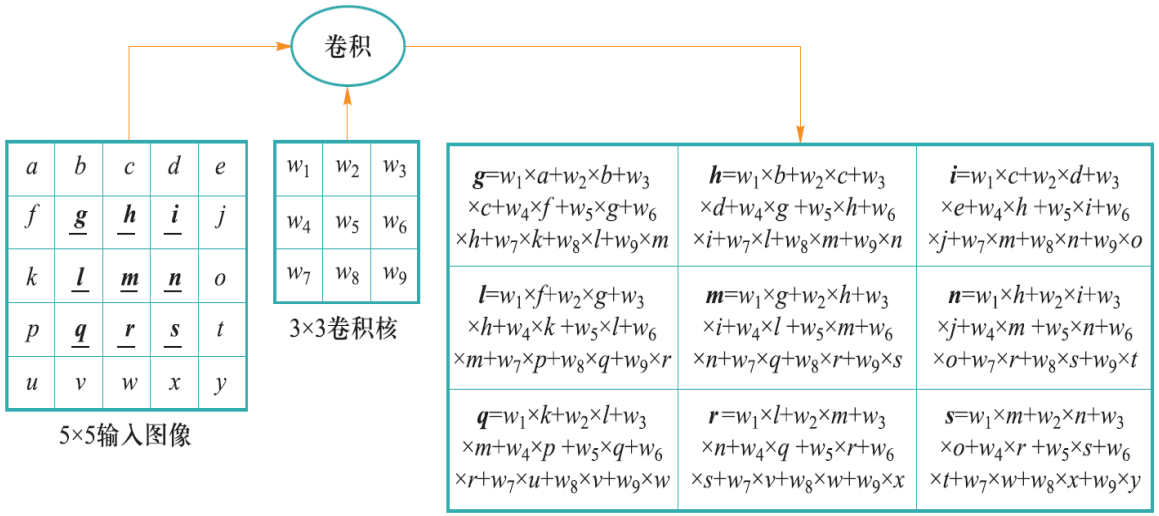


图 5.8　图像卷积例子

（注：卷积核权重在 w_i 卷积神经网络中是通过数据驱动方法学习得到的，而非手工事先指定）

卷积神经网络：卷积的意义

- 卷积这一思想利用了图像中像素点存在“空间依赖度”的特点，即相邻像素点之间具有很强的相关性、而彼此远离像素点之间的相关性很弱。
- 图像卷积计算对图像进行了下采样 (down sampling)操作。如5.8图中被卷积图像分辨率为5×5，卷积后图像的分辨率为3×3，这样被卷积图像就被“约减”了。对卷积得到的图像结果不断卷积滤波，则原始图像就会被“层层抽象、层层约减”，从而使得蕴涵在图像中的重要信息“显山露水”
- 卷积核中的权重系数w_i是通过数据驱动机制学习得到，其用来捕获图像中某像素点及其邻域像素点所构成的特有空间模式。一旦从数据中学习得到权重系数，这些权重系数就刻画了图像中像素点构成的空间分布不同模式。
- 图像卷积操作符合“视觉系统信息分层处理”这一机制，即视觉感知是由低层细胞到高层细胞对原始输入信息不断抽象完成，更高层细胞拥有更高级的感受野，并且对一些偏移、旋转等具有一定的不变性。比如，无论如何旋转手机，给出不同旋转状态的手机图像，人脑总是可以从这些不同旋转角度图像中辨认出手机这一语义对象

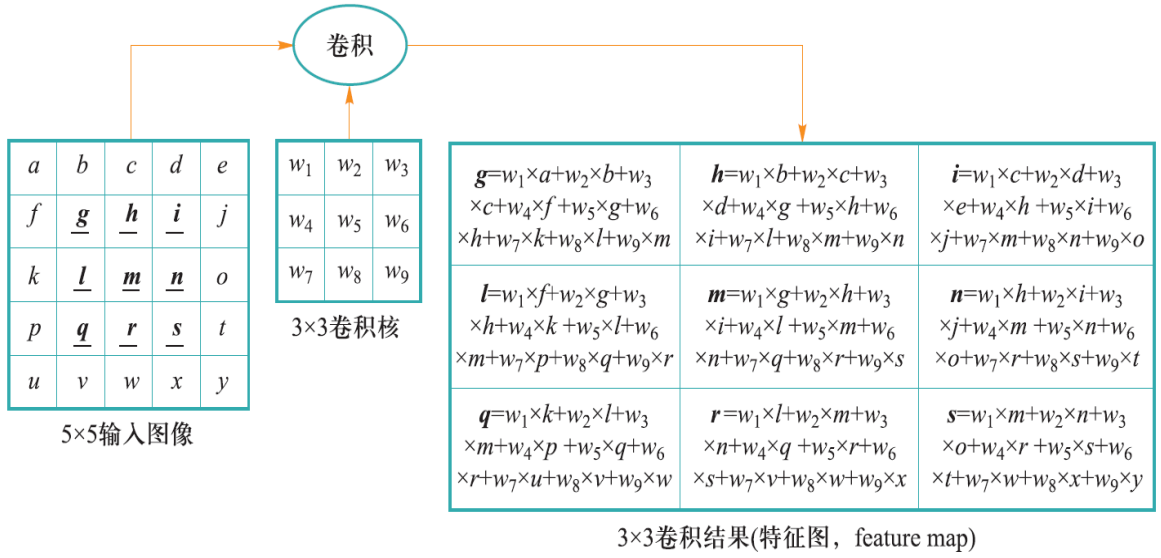


图 5.8　图像卷积例子

（注：卷积核权重在 w_i 卷积神经网络中是通过数据驱动方法学习得到的，而非手工事先指定）

卷积神经网络：不同卷积算子卷积效果

- 图5.9给出了同一幅图像经过两个不同的7 × 7高斯卷积核进行卷积滤波的结果。高斯卷积核中的权重系数具有如下特点：卷积核中心位置权重系数的取值最大，从中心位置向四周边缘扩散过程中，权重系数的取值逐渐减少。在高斯卷积核中，所有权重系数累加之值为1。
- 从图5.9可见，如果卷积核中心位置的权重系数越小且与其它卷积权重系数差别越小，则卷积所得到图像滤波结果越模糊，这被称为图像平滑操作。图中右下图与右上图相比，显得模糊，其原因在于右下图所对应的卷积核权重系数相差不大，使得像素点所对应卷积滤波结果之间彼此相差不大。
- 可以想象，在卷积过程中，被卷积图像子块中心位置像素点所对应的卷积权重系数小且与其它卷积权重系数差别不大，则意味着该像素点色度值与其近邻像素点色度值之间差距在卷积后将减少，该像素点与其近邻像素点之间的色度值也将趋同，从而使得图像变“模糊”，这很好利用了图像相邻像素点之间存在“空间依赖”的特点。

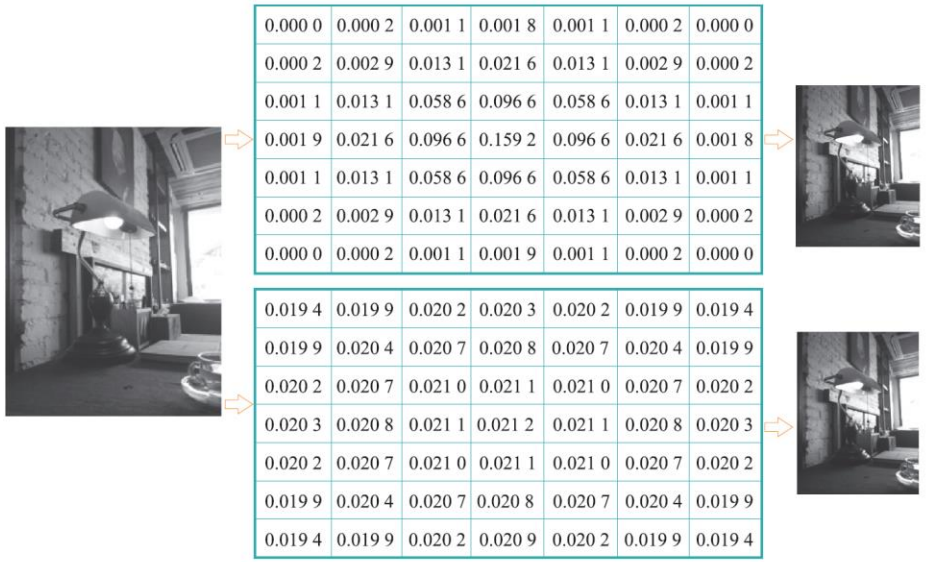


图 5.9　同一幅图像被两个不同卷积核计算的结果

卷积神经网络：填充（padding）和步长（striding）

- 填充：从前面的介绍可知，位于被卷积图像边缘位置的像素点无法形成一个以其为中心、与卷积核大小一致的图像子块区域，因此无法对边缘位置的像素点进行卷积滤波。为了使边缘位置的图像像素点也参与卷积滤波，填充技术被提出。该方法在边缘像素点周围填充“0”（即0填充），使得可以以边缘像素点为中心形成与卷积核同样大小的图像子块区域。注意，在这种填充机制下，卷积后的图像分辨率将与卷积前的图像分辨率一致，不存在下采样。
- 步长：在进行卷积操作时，通常希望被卷积所得的图像分辨率与卷积前的图像分辨率相比逐渐减少，即图像被约减。步长方法通过改变卷积核在被卷积图像中移动步长的大小来跳过一些像素，进行卷积滤波。当stride = 1时，卷积核滑动跳过1个像素，这是最基本的单步滑动，也是标准的卷积模式（如图5.8中的示例所示）。stride = *k*表示卷积核移动跳过的步长是*k*。对于图5.8中的卷积操作，如果将步长改为2，则可得到一个2 × 2的卷积滤波图像结果（见图5.10）。

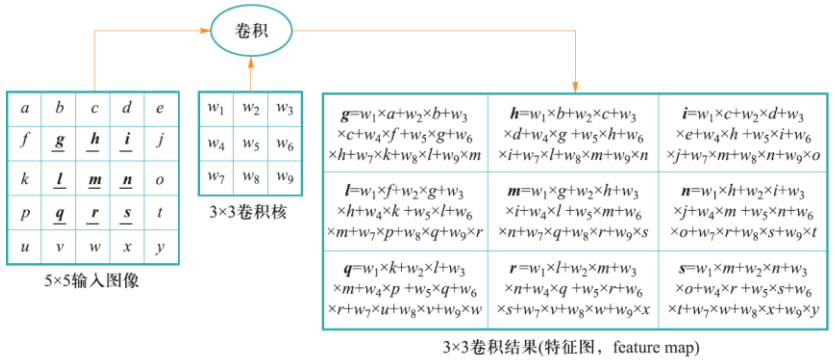


图 5.8　图像卷积例子

（注：卷积核权重在 w_i 卷积神经网络中是通过数据驱动方法学习得到的，而非手工事先指定）

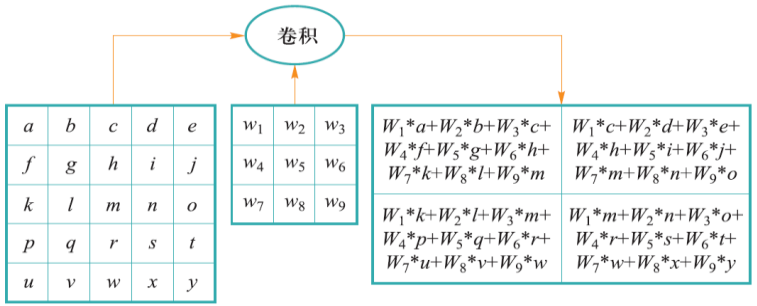


图 5.10　卷积步长为 2 时的卷积结果

卷积神经网络：卷积算子特点

- **选择性感受野**：神经科学家发现，人的视觉神经细胞对不同的视觉模式具有特征选择性（Feature Selectivity），即不同视觉神经细胞对边界、运动和颜色等不同信息具有强弱不同的选择性。因此，不同卷积核可被用来刻画视觉神经细胞对外界信息感受时的不同选择性。同时也可以看到，卷积所得结果中，每个输出点的取值仅依赖于其在输入图像中该点及其邻域区域点的取值，而与这个区域之外的其他点取值均无关，该区域被称为感受野（receptive field），正所谓“管中窥豹、见微知著”。在卷积神经网络中，感受野是卷积神经网络每一层输出的特征图（feature map）上的像素点在输入图像上映射的区域大小。也就是说，感受野是特征图上一个点对应输入图像上的区域。

- **局部感知、参数共享**：值得注意的是，卷积操作是一个线性变换（线性加权累加），但是卷积操作中的权重参数可学习、可被重复使用，从而更适合数据中蕴含的实际空间模式且减少了参数总数，这种“局部感知、参数共享”的特点减少了网络参数，一定程度地防止过拟合。

- **下采样约减抽象**：假设被卷积图像大小为$w \times w$、卷积核大小为$F \times F$、上下左右四个边缘填充像素行/列数为$\mathrm{P} = [F/2]$、步长为S，则被卷积结果的分辨率是$\frac{W-F+2P}{S} + 1$。

卷积神经网络：池化

- **最大池化（max pooling）：从输入特征图的某个区域子块中选择值最大的像素点作为最大池化结果（见图5.12）。**
- **平均池化（average pooling）：计算区域子块所包含所有像素点的均值，将均值作为平均池化结果。**
- **k-max池化（k-max pooling）：对输入特征图区域子块中的像素点取前k个最大值。如图5.13所示，从包含4个取值的每一列中选取前2个最大值就得到了k-max池化结果。**

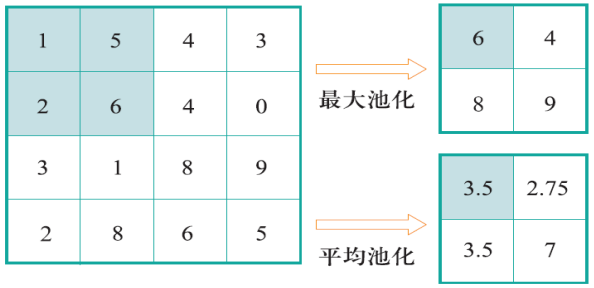


图 5.12　最大池化与平均池化操作示意

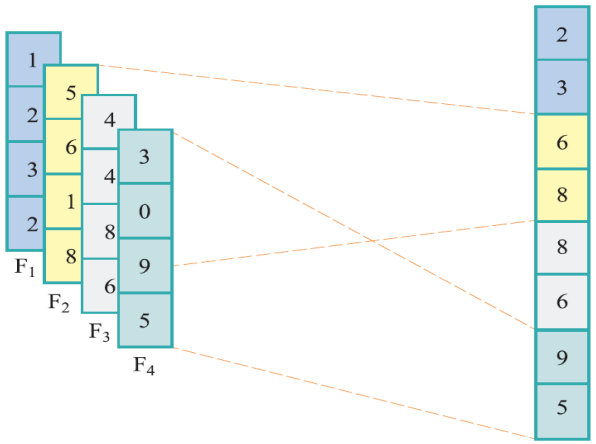


图 5.13　k–max 池化操作示意

卷积神经网络全貌

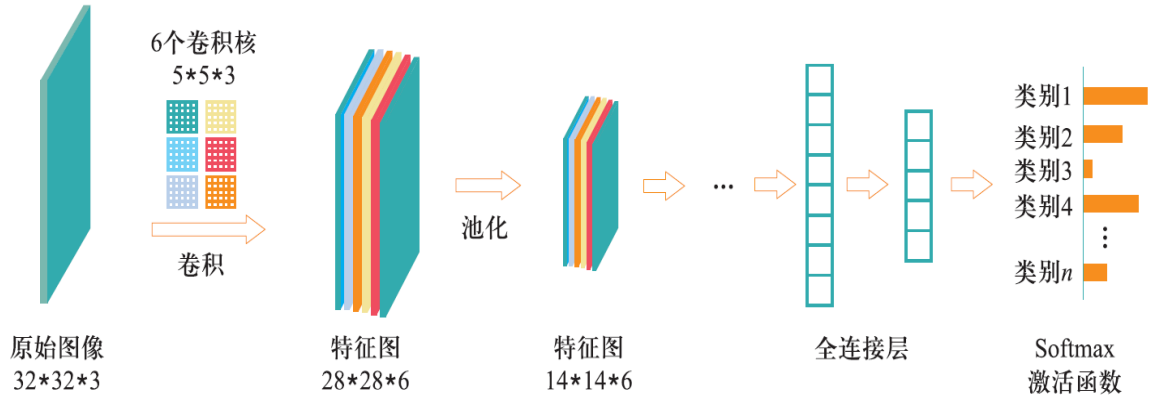


图 5.14　基于卷积神经网络的图像分类示意

如图5.14所示，对于32 × 32 × 3的彩色图像，先用6个5 × 5 × 3的卷积核对其进行卷积操作（步长为1，无填充），得到6个28 × 28的卷积结果（特征图），然后以2 × 2大小进行池化，得到6个14 × 14大小的特征图。最后通过若干个全连接层得到原始图像的向量表达，再对这个向量表达通过softmax函数进行分类识别，得到输入图像隶属于n个类别的概率。

循环神经网络

- 循环神经网络（Recurrent Neural Network, RNN）是一类处理序列数据（如文本句子、视频帧等）时所采用的网络结构。先前所介绍的前馈神经网络或卷积神经网络所需要处理的输入数据一次性给定，难以处理存在前后依赖关系的数据。
- 1990年，Jeffrey L.Elman提出循环神经网络的最初概念框架，循环神经网络的本质是希望模拟人所具有的记忆能力，在学习过程中记住部分已经出现的信息，并利用所记住的信息影响后续结点输出。循环神经网络在自然语言处理，例如语音识别、情感分析、机器翻译等领域有重要应用。
- 图5.15展示了循环神经网络的结构图。由“循环”两字可知，循环神经网络在处理数据过程中构成了一个循环体。对于一个序列数据，在每一时刻t，循环神经网络单元会读取当前输入数据x_t和前一时刻输入数据x_{t-1}所对应的隐式编码结果h_{t-1}，一起生成t时刻的隐式编码结果h_t。接着将h_t后传，去参与生成$t+1$时刻输入数据x_{t+1}的隐式编码h_{t+1}。如此循环处理，直至该序列数据被处理完毕。

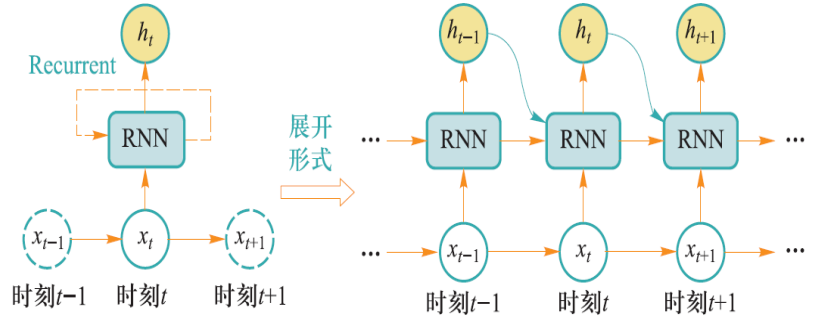


图 5.15　循环神经网络结构及其展开形式示意

时刻t所得到的隐式编码h_t是由上一时刻隐式编码h_{t-1}和当前输入x_t共同参与生成的，这可认为隐式编码h_{t-1}已经“记忆”了t时刻之前的时序信息，或者说前序时刻信息影响了后续时刻信息的处理

循环神经网络

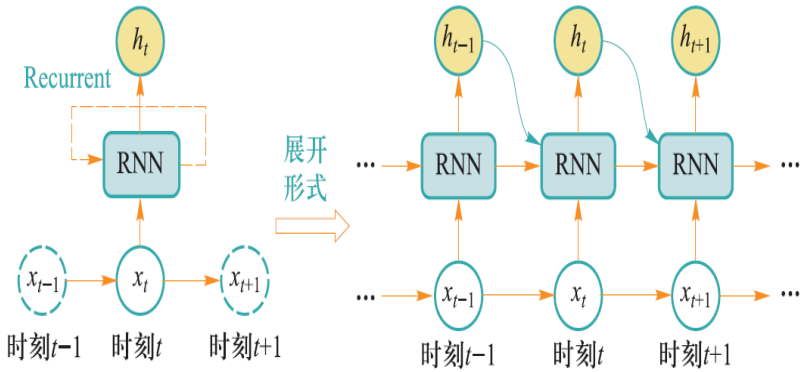


图 5.15　循环神经网络结构及其展开形式示意

$$h_t = \Phi\left(U \times x_t + W \times h_{t-1}\right) = \Phi\left(U \times x_t + W \times \Phi\left(U \times x_{t-1} + W \times h_{t-2}\right)\right)$$

$$= \Phi\left(U \times \underbrace{x_t}_{t\text{ 时刻输入}} + W \times \Phi\left(U \times \underbrace{x_{t-1}}_{t-1\text{ 时刻输入}} + W \times \Phi\left(U \times \underbrace{x_{t-2}}_{t-2\text{ 时刻输入}} + \cdots\right)\right)\right)$$

在时刻t，一旦得到当前输入数据x_t，循环神经网络会结合前一时刻$t-1$得到的隐式编码h_{t-1}，如下产生当前时刻隐式编码h_t：

$$h_t = \Phi(\mathrm{U} \times x_t + \mathrm{W} \times h_{t-1})$$

这里$\Phi(\cdot)$是激活函数，一般可为Sigmoid或者Tanh激活函数，U与W为模型参数。从这里可看出，当前时刻的隐式编码输出h_t不仅仅与当前输入数据x_t相关，与网络已有的“记忆”h_{t-1}也有着密不可分的联系。

循环神经网络：处理自然语言的例子

- 下面以自然语言例子来解释循环神经网络是如何处理时序数据。假设希望将自然语言句子进行分类，分类为饮食、体育、文化等不同类别。首先，对文本语句进行分词操作，如图5.16所示，循环神经网络在读入每个单词后，产生对应单词的隐式编码，并利用其来参与后续单词的隐式编码。
- 一旦得到了每个单词的隐式编码后，通常可以通过如下方法得到一个句子的向量编码：(a)考虑到句子中所有单词的信息均被后向传递，因此将最后一个单词的输出作为整个句子向量编码表示；(b)将每个单词的隐式编码进行加权平均，将加权平均结果作为句子的向量编码表示。这样，一旦得到了句子的向量编码表示，就可以利用某个分类器，对句子的类别进行判断。

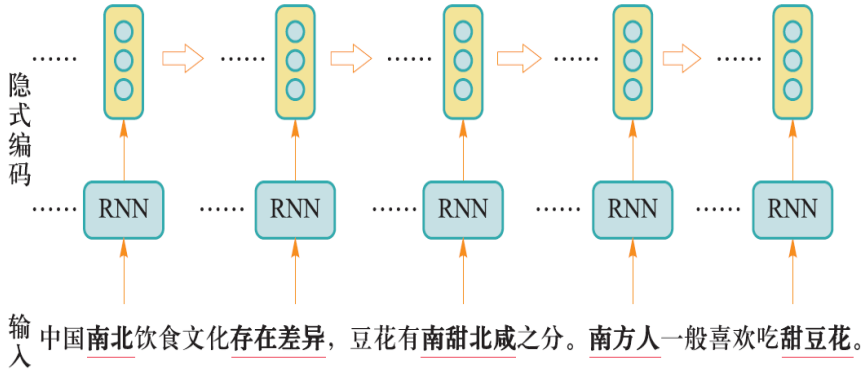


图 5.16　循环神经网络的应用示例

“南北”“存在差异”“南甜北咸”“南方人”“甜豆花”等单词之间存在前后依赖，这种依赖会被有效利用起来，得到各单词的隐式编码及句子的向量表达。

循环神经网络：应用模式

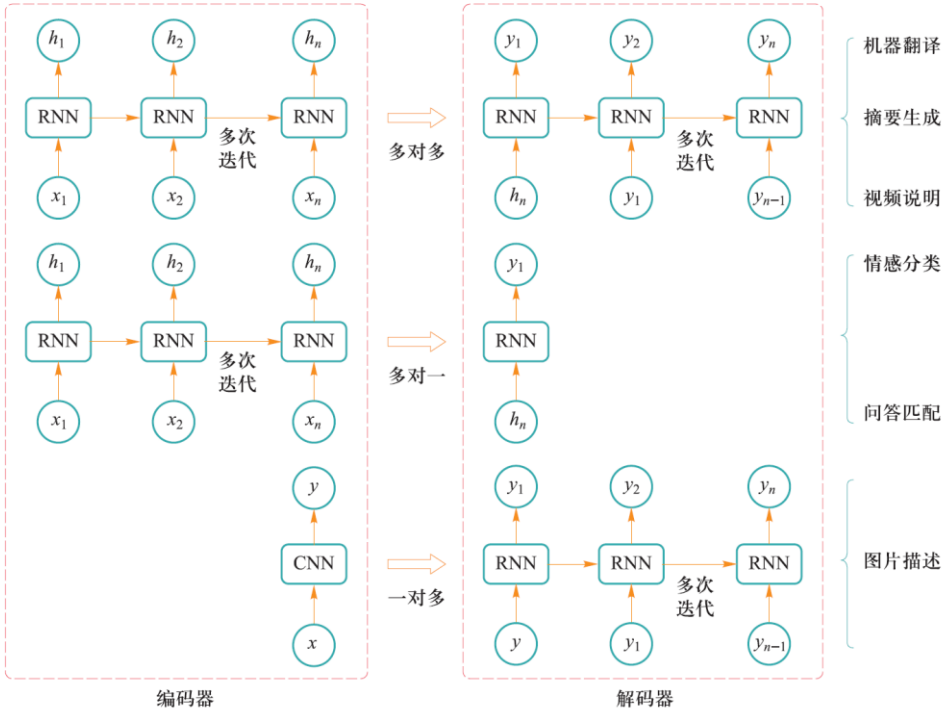


图 5.17　输入输出不同情况下循环神经网络结构示例

在循环神经网络中，根据输入序列数据与输出序列数据中所包含“单元”的多寡，循环神经网络可以实现“多对多”（即输入和输出序列数据中包含多个单元，常用于机器翻译）、“多对一”（即输入序列数据包含多个单元，输出序列数据只包含一个单元，常用于文本的情感分类）和“一对多”（即输入序列数据包含一个单元，输出序列数据包含多个单元，常用于图像描述生成）三种模式

循环神经网络：梯度传递

- 循环神经网络虽然具有一定的记忆能力，但当彼此相关信息在序列中相距较远时，循环神经网络无法捕捉到数据之中的时序依赖关系。除此之外，当输入序列过长时，循环神经网络也容易出现梯度消失（gradient vanishing）或者梯度爆炸（gradient exploding）的问题。
- 图5.18以词性标注为例子给出了一个输入序列数据$(x_1, \cdots, x_{t-1}, x_t, x_{t+1}, \ldots, x_T)$被循环神经网络处理的示意图。
- 在图5.18中，时刻t输入数据x_t（文本中的单词）的隐式编码为h_t、其真实输出为y_t（单词词性）、模型预测x_t的词性是O_t。参数W_x将x_t映射为隐式编码h_t、参数W_o将h_t映射为预测输出O_t、h_{t-1}通过参数W_h参与h_t的生成。图中W_x、W_o和W_h是复用参数。
- 假设时刻t隐式编码如下得到：$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$。使用交叉熵损失函数计算时刻$t$预测输出与实际输出的误差$E_t$。显然，整个序列产生的误差为$E = \frac{1}{2}\sum_{t=1}^{T} E_t$。

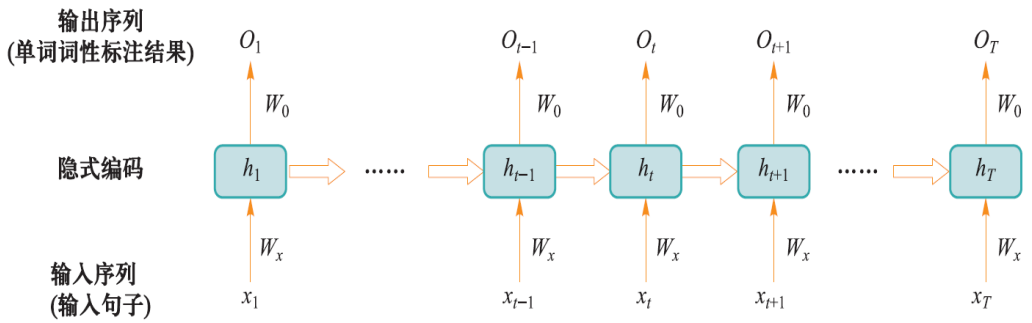


图 5.18　循环神经网络中梯度传递示意

循环神经网络：更新参数

下面介绍如何根据时刻t所得误差来更新参数W_x。在时刻t计算所得O_t不仅涉及到了时刻t的W_x，而且也涉及了前面所有时刻的W_x，按照链式求导法则，E_t在对W_x求导时候，也需要对前面时刻的W_x依次求导，然后再将求导结果进行累加，即：

$$\frac{\partial E_t}{\partial W_x}=\sum_{i=1}^{t}\frac{\partial E_t}{\partial O_t}\frac{\partial O_t}{\partial h_t}\left(\prod_{j=i+1}^{t}\frac{\partial h_j}{\partial h_{j-1}}\right)\frac{\partial h_i}{\partial W_x}$$

$$\text{其中}\prod_{j=i+1}^{t}\frac{\partial h_j}{\partial h_{j-1}}=\prod_{j=i+1}^{t}tanh'\times W_h$$

令$t=3$，则有：

$$\frac{\partial E_3}{\partial W_x}=\frac{\partial E_3}{\partial O_3}\frac{\partial O_3}{\partial h_3}\frac{\partial h_3}{\partial W_x}+\frac{\partial E_3}{\partial O_3}\frac{\partial O_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial W_x}+\frac{\partial E_3}{\partial O_3}\frac{\partial O_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W_x}$$

由于$tanh$函数的导数取值位于0到1区间，对于长序列而言，若干多个0到1区间的小数相乘，会使得参数求导结果很小，引发梯度消失问题。E_t对W_h的求导类似，这里就不列出了。为了缓解梯度消失问题，长短时记忆模型（Long Short-Term Memory，LSTM）被提出。

循环神经网络：长短时记忆模型

- 与简单的循环神经网络结构不同，长短时记忆网络中引入了内部记忆单元（internal memory cell）和门（gates）结构来对当前时刻输入信息以及前序时刻所生成信息进行整合和传递。在这里，内部记忆单元中信息可视为对“历史信息”的累积。
- 常见的LSTM模型中有输入门(input gate)、遗忘门(forget gate)和输出门(output gate)三种门结构。对于给定的当前时刻输入数据x_t和前一时刻隐式编码h_{t-1}，输入门、遗忘门和输出门通过各自参数对其编码，分别得到三种门结构的输出i_t、f_t和o_t。
- 在此基础上，再进一步结合前一时刻内部记忆单元信息c_{t-1}来更新当前时刻内部记忆单元信息c_t，最终得到当前时刻的隐式编码h_t

循环神经网络：长短时记忆模型

表 5.4　长短时记忆网络中所用符号及其含义

符号	内容描述或操作
x_t	时刻 t 的输入数据
i_t	输入门的输出：$i_t=\text{sigmoid}(W_{xi}x_t+W_{hi}h_{t-1}+b_i)$ （W_{xi}、W_{hi} 和 b_i 为输入门的参数）
f_t	遗忘门的输出：$f_t=\text{sigmoid}(W_{xf}x_t+W_{hf}h_{t-1}+b_f)$ （W_{xf}、W_{hf} 和 b_f 为遗忘门的参数）
o_t	输出门的输出：$o_t=\text{sigmoid}(W_{xo}x_t+W_{ho}h_{t-1}+b_o)$ （W_{xo}、W_{ho} 和 b_o 为输出门的参数）
c_t	内部记忆单元的输出：$c_t=f_t\odot c_{t-1}+i_t\odot\tanh(W_{xc}x_t+W_{hc}h_{t-1}+b_c)$ （注：W_{xc}、W_{hc} 和 b_c 为记忆单元的参数。输入门 i_t 控制有多少信息流入当前时刻内部记忆单元 c_t，遗忘门控制上一时刻内部记忆单元 c_{t-1} 中有多少信息可累积到当前时刻内部记忆单元 c_t）
h_t	时刻 t 输入数据的隐式编码：$h_t=o_t\odot\tanh(c_t)=o_t\odot\tanh(f_t\odot c_{t-1}+i_t\odot\tanh(W_{xc}x_t+W_{hc}h_{t-1}+b_c))$ （注：输入门、遗忘门和输出门的信息 i_t、f_t、o_t 一起参与得到 h_t）
$\odot$	两个向量中对应元素按位相乘（element-wise product）。例如，[10 6 3 7] ⊙ [0.2 0.1 0.8 0.5] = [2 0.6 2.4 3.5]

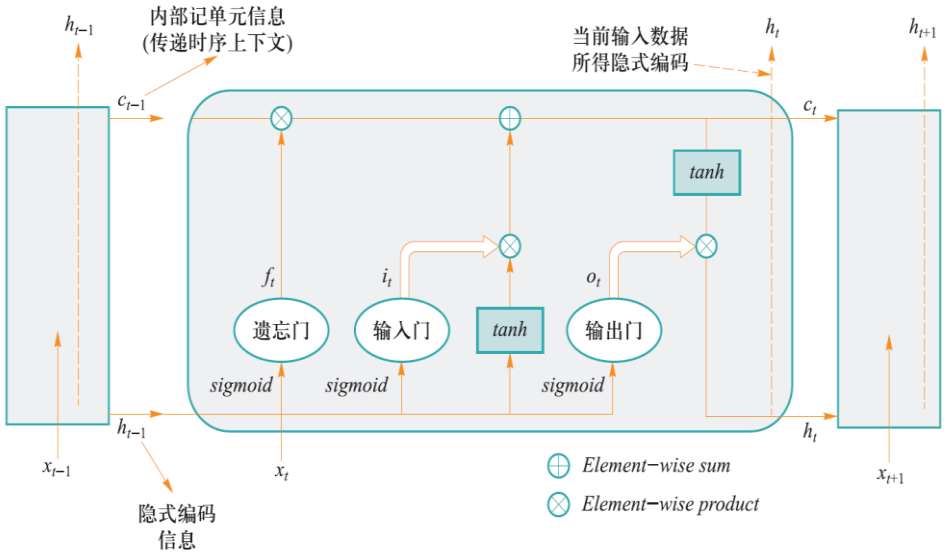


图 5.19　长短时记忆网络模型

（注：在每个时刻 t，只有内部记忆单元信息 c_t 和隐式编码 h_t 这两种信息起到了对序列信息进行传递的作用；只有 h_t 作为本时刻的输出用于分类等处理）

输入门、遗忘门和输出门通过各自参数对当前时刻输入数据x_t和前一时刻隐式编码h_{t-1}处理后，利用$sigmoid$对处理结果进行非线性映射，因此三种门结构的输出i_t、f_t和o_t值域为$(0,1)$。正是由于三个门结构的输出值为位于0到1之间的向量，因此其在信息处理中起到了“调控开关”的“门”作用。三个门结构所输出向量的维数、内部记忆单元的维数和隐式编码的维数均相等。

循环神经网络：长短时记忆模型

- 以输入门$i_t = sigmoid(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$为例，在$sigmoid$函数作用下，假设输入门$i_t$的取值为$(0.05, 0.1, 0.8)$，则通过$i_t \odot tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$中按位相乘$\odot$操作，输入门会“阻断”一些信息进行后续操作（如0.05和0.1所对应按位相乘的信息）、也会“放入”一些信息进行后续操作（如0.8所对应按位相乘的信息）。当然，如果门的某一位取值为0，表示这一位所对应信息为全闭、禁止信息通过；如果门的某一位取值为1，表示这一位所对应的信息为全开、容许信息全部通过。遗忘门不容许一些信息后续传递，从另外一个侧面而言是为了关注需要关注信息。

- 从$c_t = f_t \odot c_{t-1} + i_t \odot tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$可以看出，对当前时刻$t$所对应内部记忆单元中信息的调整涉及到遗忘门信息、$t-1$时刻所对应内部记忆单元中信息、输入门信息和$t-1$时刻的隐式编码。

- 由于$sigmoid$函数的值域在$(0,1)$之间，输入门、遗忘门和输出门三种门结构采用$sigmoid$这一非线性映射函数而起到了信息“控制门”的作用。在内部记忆单元和隐式编码中，使用了$tanh$这一非线性映射函数而没有继续使用$sigmoid$非线性映射函数，其原因在于$tanh$函数的值域为$(-1,1)$，使得$tanh$函数在进行信息整合时可起到信息“增（为正）”或“减（为负）”的效果。

循环神经网络：长短时记忆模型

在介绍循环神经网络时，谈及到了根据误差来更新参数存在梯度消失问题，下面来看看LSTM如何避免了梯度消失问题。

对于$c_t = f_t \odot c_{t-1} + i_t \odot tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$，有如下求导结果的存在：

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t + \frac{\partial f_t}{\partial c_{t-1}} \times c_{t-1} + \cdots\cdots$$

可见，$\frac{\partial c_t}{\partial c_{t-1}}$求导的结果至少大于等于$f_t$，即遗忘门的输出结果。如果遗忘门选择保留旧状态，则这一求导结果就接近1（或者接近向量1），使得梯度是存在的，从而避免了梯度消失问题。也就是说，LSTM通过引入门结构，在从t到$t+1$过程中引入加法来进行信息更新，避免了梯度消失问题。

整体来看，内部记忆单元信息c_t和隐式编码h_t这两种信息起到了对序列信息进行传递的作用。内部记忆单元信息c_t好比人脑的长时记忆、隐式编码h_t代表了短时记忆。

注意力机制

- 注意指意识对一定信息或对象的指向与集中的过程，具有选择性和集中性等特点，是一切心理过程得以产生和进行的必不可少的心理属性。
- 在自然语言中，注意力可理解为句子级或篇章级中单词和单词之间因为上下文而共同出现的概率；在图像分析中，注意力可理解为在一定视觉区域中视觉子块之间的空间分布模式；在机器翻译中，注意力可理解为输入源语言单词和输出目标语言单词之间的关联。
- 谷歌公司的研究人员于2017年提出了Transformer模型，引入“自注意力（self-attention）”来计算自然语言句子中单词和单词之间的概率关联（Ashish et al.，2017）。
- 图5.21给出了由四个单词组成的句子“项庄舞剑、意在沛公”中单词“意在”与句子中其他单词之间概率关联的计算示意。

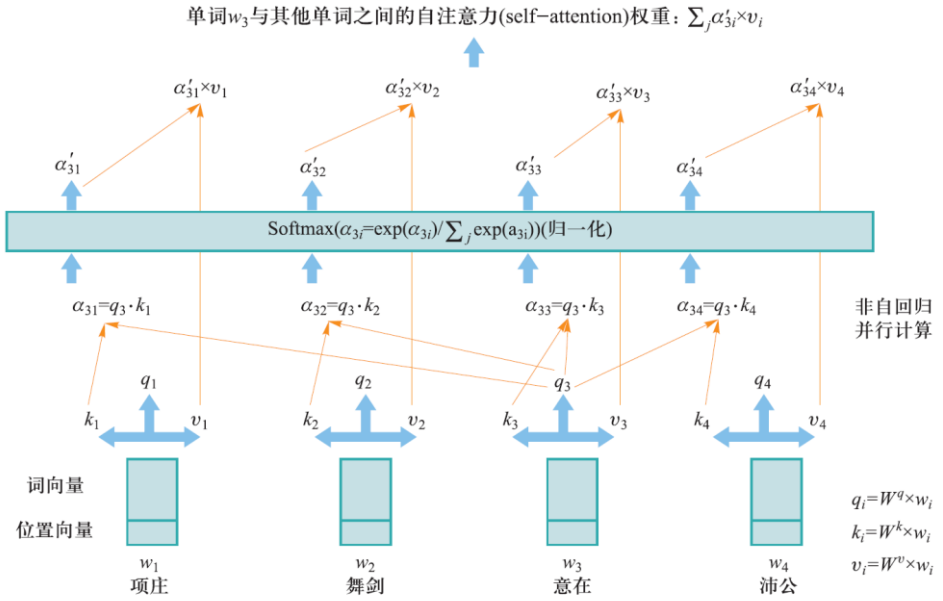


图 5.21　句子中每个单词与其他单词自注意力取值计算示意（以单词 w_3 为例）

注意力机制

- 首先生成每个单词的内嵌向量（包含了单词在句子中位置编码向量信息），记为$w_i(1 \le i \le 4)$，如下计算每个单词w_i的查询向量（query）、键向量(key)和值向量(value):
- 查询向量：$q_i = W^q \times w_i$；键向量：$k_i = W^k \times w_i$；值向量：$v_i = W^v \times w_i$
- 后面可以看到，对每个单词而言w_i，W^q、W^k和W^v三个映射矩阵都是一样的，也是自注意力模型需要训练的全部参数。
- 自注意力模型就是要挖掘单词w_i与其他单词在句子中因为上下文（context）关联而具有的自注意力取值大小。以图5.21中单词w_3与其他单词之间自注意力取值大小计算为例:

1、计算w_3所对应查询向量与其他单词健向量之间的点积，该点积可作为单词w_3与其他单词之间的关联度：$\alpha_{3i} = q_3 \cdot k_i$

2、对α_{3i}结果通过softmax函数进行归一化操作，得到a'_{3i}。对于给定“项庄舞剑、意在沛公”句子，a'_{3i}记录了“意在”单词与其他单词所具有的重要程度（即注意力）。

3、a'_{3i}乘以每个单词w_i所对应的值向量v_i，即$a'_{3i} \times v_i$。

4、对上一步结果进行$\sum_i a'_{3i} \times v_i$，这个结果作为在当前句子语境下单词$w_3$“注意”到与其他单词的关联程度（self-attention）。

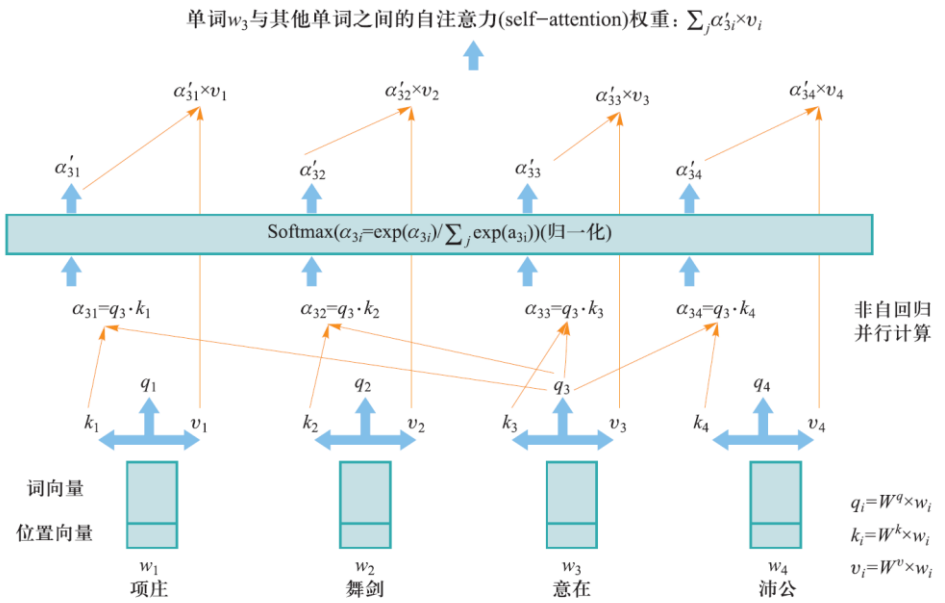


图 5.21　句子中每个单词与其他单词自注意力取值计算示意（以单词 w_3 为例）

注意力机制

- 这里的查询向量、键向量和值向量可从搜索引擎角度来理解：查询向量为搜索单词、键向量为被搜索物品的描述信息（如标题等）、值向量为被搜索物品的语义内容。于是可如下理解自注意力关联：在每个单词具有自身内容（值向量表示）的基础上，给定自然语言句子，计算给定单词和句子中其他单词之间归一化关联度，利用关联度大小来对每个单词的自身内容调整和加权取和，以反映每个单词在句子中因为上下文语义而与其他单词之间的自注意力。
- 当然，也可引入“多头”注意力（multi-headed attention）机制从更多角度来挖掘某个单词与其他单词之间概率关联。同时也应该了解，某个单词的自注意力关联可并行计算，而不像循环神经网络RNN那样，需要通过自回归的方式来一个单词接着一个单词来处理。
- 一旦计算得到单词和其他单词所形成的自注意力关联后，Transformer再将这些自注意力关联通过前向神经网络（feed-forward）进行非线性映射。非线性变换的引入能帮助模型捕捉到更丰富的特征和语义信息，有助于提高模型的表达能力（如更好捕获单词和单词之间复杂的共现概率）。

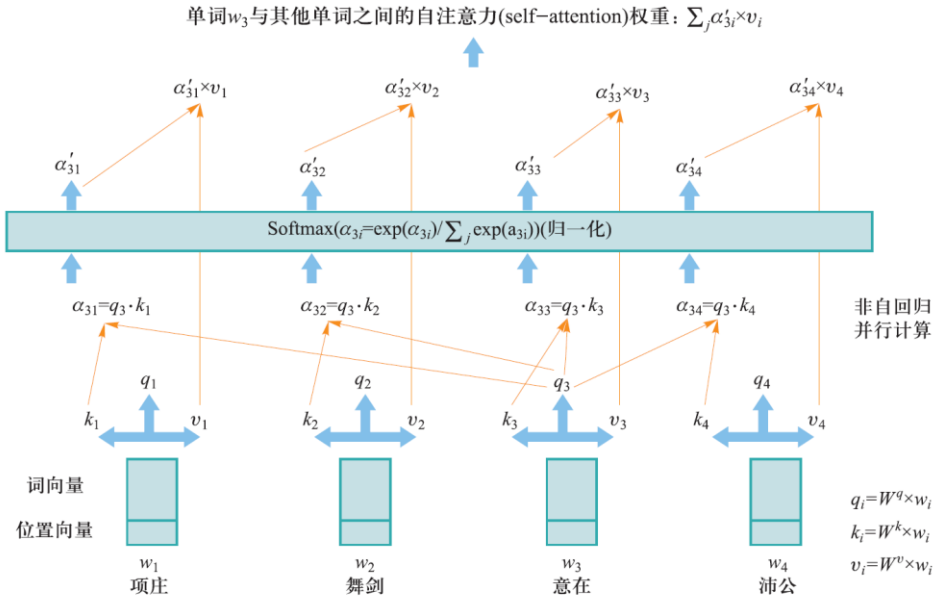


图 5.21　句子中每个单词与其他单词自注意力取值计算示意（以单词 w_3 为例）

神经网络正则化

深度神经网络结构复杂、参数众多，很容易造成过拟合（over-fitting）。为了缓解神经网络在训练过程中出现的过拟合现象，需要采取一些正则化技术来提升神经网络的泛化能力（generalization）。

- **Dropout**：指在训练神经网络的过程中随机丢掉一部分神经元来降低神经网络的复杂度，从而防止过拟合。Dropout的实现方法很简单：在每次迭代训练中，以一定概率随机屏蔽每一层中的若干神经元，用余下神经元构成的网络继续训练。
- **批归一化**：批归一化（batch normalization）就是通过规范化手段，把神经网络每层中任意神经元的输入值分布改变成均值为0、方差为1的标准正态分布，把偏移较大的分布强制映射为标准正态分布。经过批归一化处理，激活函数的输入值被映射到非线性函数梯度较大的区域，使得梯度变大从而克服梯度消失问题，进而加快收敛速度。

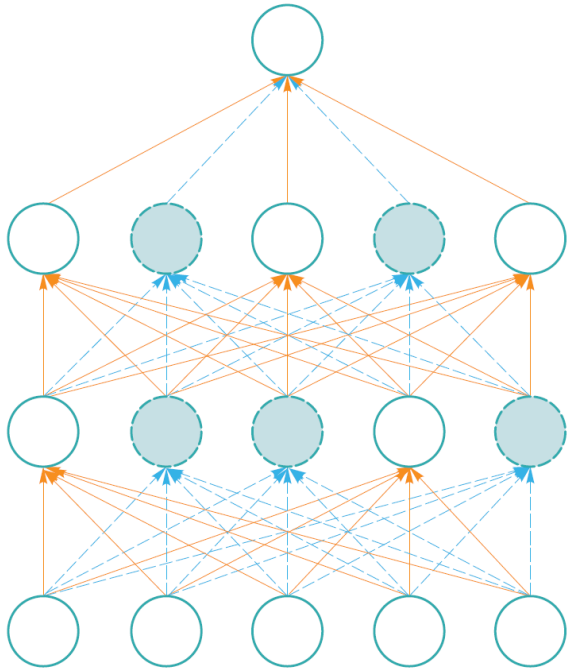


图 5.22　使用 Dropout 的神经网络模型

神经网络正则化

深度神经网络结构复杂、参数众多，很容易造成过拟合（over-fitting）。为了缓解神经网络在训练过程中出现的过拟合现象，需要采取一些正则化技术来提升神经网络的泛化能力（generalization）。

L_1和L_2正则化：对于具有n个训练数据$\{(x_1, y_1), \ldots, (x_n, y_n)\}$的神经网络，其中$(x_i, y_i)(1 \leq i \leq n)$分别为输入样本及其对应的标签。加入正则化项后，神经网络的损失函数一般可如下表示：

$$\min \frac{1}{n} \sum_{i=1}^{n} \underbrace{Loss\big(y_i, f(\boldsymbol{W}, x_i)\big)}_{\text{损失函数}} + \underbrace{\lambda}_{\text{正则化权重}} \times \underbrace{\Phi(\boldsymbol{W})}_{\text{正则化项}}$$

其中，$f(\boldsymbol{W}, x_i)$表示参数为$\boldsymbol{W}$的神经网络对输入x_i的预测、$\Phi(\boldsymbol{W})$为正则化项（又称惩罚项）、λ为正则项权重。

正则化项$\Phi(\boldsymbol{W})$一般用模型参数$\boldsymbol{W}$的范数形式来表示。假设神经网络模型中参数数目为$\mathbb{N}$，则参数的范数形式主要有：

L_0范数：数学表示为$\|\boldsymbol{W}\|_0 = \sum_{i=1}^{\mathbb{N}} \mathbb{I}[w_i \neq 0]$，其中$\mathbb{I}$是指示函数，若括号内表达式为真，函数值为1，反之为0。所以L_0范数指模型参数$\boldsymbol{W}$中非零元素个数。L_0可实现模型参数的稀疏化。但是，由于L_0范数正则化是一个NP Hard问题，难以求解，因此一般不用L_0范数。

L_1范数：数学表示为$\|W\|_1 = \sum_{i=1}^{\mathbb{N}} |w_i|$，指模型参数$\boldsymbol{W}$中各个元素的绝对值之和。$L_1$范数也被称为“稀疏规则算子”（Lasso regularization）。

L_2范数：数学表示为$\|W\|_2 = \sqrt{\sum_{i=1}^{\mathbb{N}} w_i^2}$，指模型参数$\boldsymbol{W}$中各个元素平方和的开方。

深度学习在自然语言中应用之一：词向量生成

- 在基于规则和统计的自然语言传统方法中，通常将单词视为独立符号。为了表达文本，“词袋（bag of words）”模型被采用，但是这一模型忽略了文本单词之间的依赖关系，仅仅将文本看作单词的集合（忽略了单词之间的先后次序）。
- 在词袋模型中，一个单词按照词典序被表示为一个词典维数大小的向量（称为one-hot vector），向量中该单词所对应位置按照其在文档中出现与否取值为1或0。1表示该单词在文档中出现，0表示没有出现（见图5.23）。每个单词被表达为单词向量后，一些与自然语言相关的工作（如聚类、同义词计算、主题挖掘等）可以依次开展。
- 为了刻画不同单词之间的语义相关性，研究人员希望使用一种分布式向量表达(distributed vector representation)对不同单词进行表达。利用深度学习模型，可将每个单词表征为N维实数值的分布式向量。这样一来可以把对文本内容分析简化为N维向量空间中的向量运算，如两个单词在向量空间上所计算结果可用来衡量这两个单词之间的相似度。用深度学习算法生成每个单词的向量表达，词向量（Word2Vec）是较为经典的模型

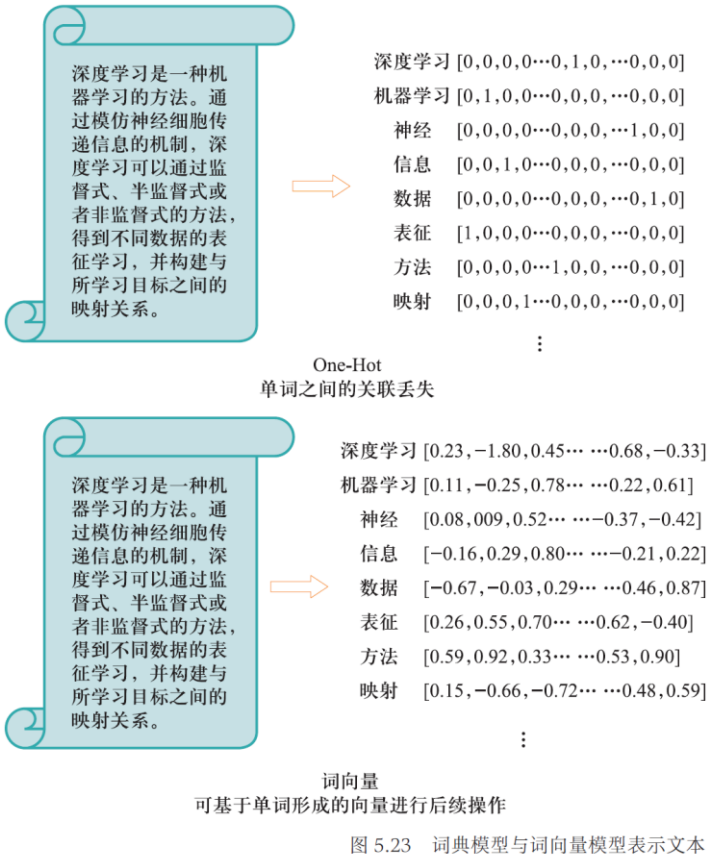


图 5.23 词典模型与词向量模型表示文本

深度学习在自然语言中应用之一：词向量生成

- 假设词典中有V个不同的单词，现在考虑如何生成第k个单词的N维词向量（如图5.24所示）。首先，将该单词表示成V维one-hot 向量$\boldsymbol{X}$，向量$\boldsymbol{X}$中第k个位置取值为1、其余位置取值均为0。隐藏层神经元大小为N，每个神经元记为$h_i(1 \le i \le N)$。向量$\boldsymbol{X}$中每个$x_i(1 \le i \le V)$与隐藏层神经元是全连接，连接权重矩阵为$W_{V\times N}$。输出层是V维归一化的概率值，其中y_k对应第k个单词归一化的概率值，显然其取值应该远大于其它输出所对应的归一化概率值。
- 在图5.24中，输入层中x_k与隐藏层连接权重为$\{w_{k1}, w_{k2}, \cdots, w_{kN}\}$，输入单词的$N$维隐式表达$\boldsymbol{h} = \boldsymbol{W}^{\boldsymbol{T}} \times \boldsymbol{X} = v_{wk}^T$。所得到的隐藏层结果与输出层是全连接，全连接权重矩阵为$W'_{N\times V}$。从图中可知，第k个单词对应的输出为 $y_k = (v_{wk}^T)_{1\times N} \times (W'_{(:,k)})_{N\times 1}$。这里$W'_{(:,k)}$指的从连接权重矩阵$W'_{N\times V}$中取出第$k$列的向量。接着对 y_k通过Softmax函数进行归一化，得到第k个单词对应的归一化概率输出。显然，第k个单词归一化的概率值应该远远大于输出层中其他位置所对应的归一化概率值。
- 这里$W_{V\times N}$和$W'_{N\times V}$为模型参数。一旦训练得到了图5.24的神经网络，就可以将输入层中x_k与隐藏层连接权重为$\{w_{k1}, w_{k2}, \cdots, w_{kN}\}$作为第$k$个单词$N$维词向量(word vector)。

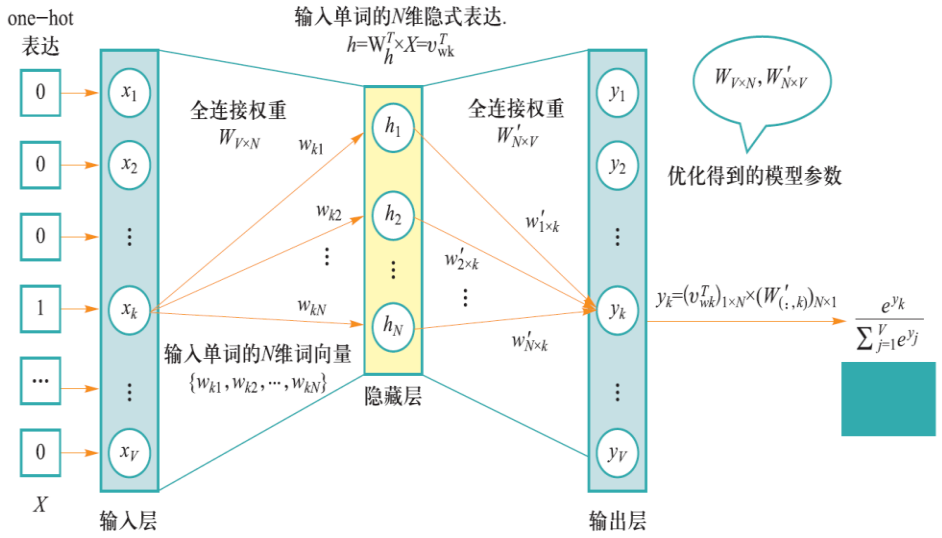


图 5.24　单个单词的词向量生成示意

深度学习在自然语言中应用之一：词向量生成

- 假设词典中有V个不同的单词，现在考虑如何生成第k个单词的N维词向量（如图5.24所示）。首先，将该单词表示成V维one-hot 向量$\boldsymbol{X}$，向量

$$\text{word2vec model} = max\text{P}(\text{输出表达}|\text{输入表达})$$

$$= max\log\left(\frac{e^{y_k^*}}{\sum_{j=1}^{V} e^{y_j}}\right) = max(y_k^* - log\sum_{j=1}^{V} e^{y_j})$$

其中星号（$*$）表示预测值。对上式取其相反数，令其最小，则得到损失函数:

$$loss = -y_k^* + log\sum_{j=1}^{V} e^{y_j}$$

这样就可以利用梯度下降和误差后向传播来优化训练参数$W_{V\times N}$和$W'_{N\times V}$了。

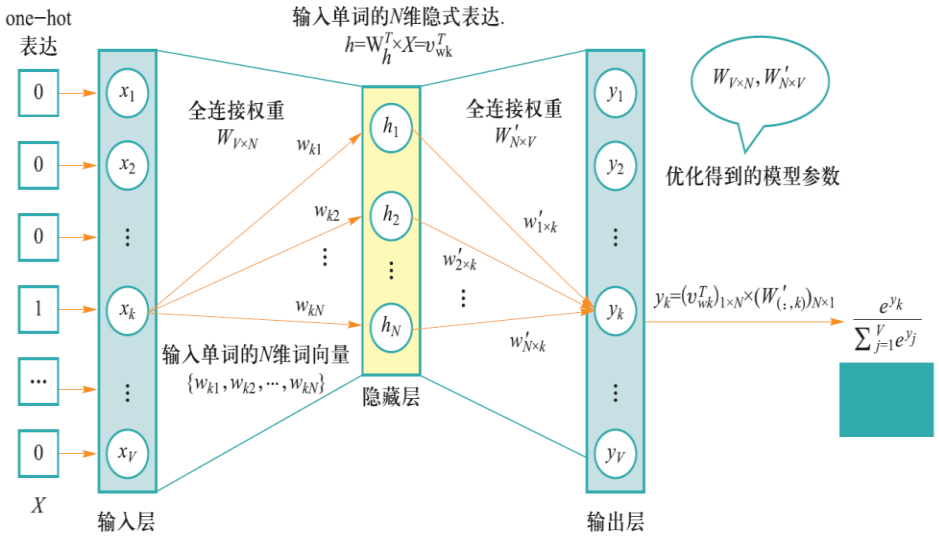


图 5.24　单个单词的词向量生成示意

- 通常Word2Vec的模型参数有两种训练模式。一种是Continuous Bag-of-Words (CBoW)，即根据某个单词所处的上下文单词来预测该单词。另一种是Skip-gram，即利用某个单词来分别预测该单词的上下文单词。

深度学习在自然语言中应用之一：词向量生成

- 图5.25给出了基于CBoW来得到词向量的示意图。给出包含c个单词的句子$\{x_1, \ldots, x_{k-1}, x_k, x_{k+1}, \ldots, x_c\}$。在CBoW中，可用$x_k$的前序$k-1$个单词和后续单词$c-k$个单词一起来预测$x_k$。每个输入单词仍然是$V$维向量（可以是one-hot的表达），隐式编码为每个输入单词所对应编码的均值$\boldsymbol{h} = \frac{1}{c-1}\boldsymbol{W^T} \times (X_1 + \cdots + X_{k-1} + X_{k+1} + \cdots + X_c)$。
- 对于计算得到的隐式编码，要求其输出归一化概率中，单词x_k所对应的y_k取值最大，即用x_k的上下文来预测x_k。定义好相应的损失函数，可训练得到参数$W_{V\times N}$和$W'_{N\times V}$。

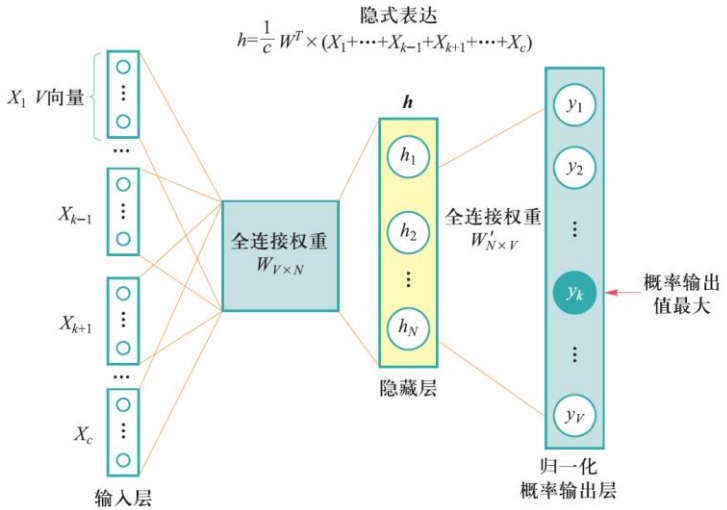


图 5.25　基于 CBoW 模型的单词词向量生成示意

深度学习在自然语言中应用之一：图像分类与目标定位

- 在图像分类和目标定位过程中，单次目标检测模型不仅要检测物体位置，同时它也要判断所检测物体的类别标签。因此，图像分类和目标定位需要同时完成回归问题（位置判断）与分类问题（物体识别）两个任务，这种同时解决两个及两个以上问题的任务为多任务学习（Multi-Task Learning）。
- 输入一幅图像，首先利用卷积神经网络来提取视觉特征。在图5.26中，给定一幅图像，卷积神经网络经过若干次卷积与池化的操作，可提取一个特征向量来表示输入图像。这个向量作为输入特征分别传送给两个任务，即分类任务和定位任务：a)分类任务。Softmax分类函数将全连接层所得特征向量映射为归一化概率值，用来表示每个概念在输入图像中出现的概率。在Softmax分类函数的输出概率向量中，向量中每一维代表输入图像中某个类别出现的概率。在训练中，可用交叉熵等损失函数来计算模型学习的误差；b)目标定位。将全连接层输出向量转换为一个四维向量，四维向量分别代表一个包围盒的左上角横坐标和纵坐标以及该矩形框宽与高取值，并使用均方差损失函数计算模型的误差。最终，将两个任务所得误差相加，作为整个模型的误差，用于计算模型梯度，更新模型参数。这样，训练好的模型可以同时对图像分类以及对图像中视觉目标对象进行定位。

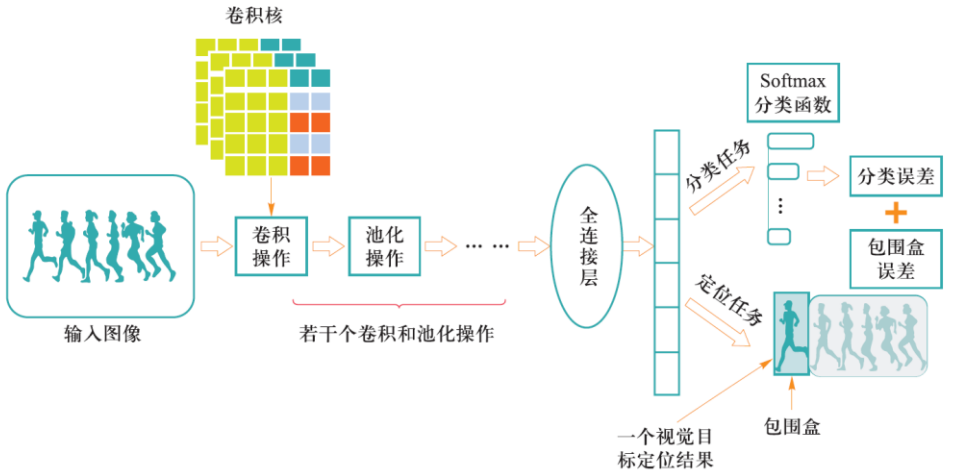


图 5.26　利用卷积神经网络进行图像分类与目标定位示意

延伸阅读

爬上树梢与攀登月球?

《炼金术与人工智能》（Alchemy and Artificial Intelligence）：1965年12 月，先后在MIT和伯克利任教的休伯特 德雷福斯（Hubert Lederer Dreyfus）教授以兰德公司顾问的身份，发表了《炼金术与人工智能》（Alchemy and Artificial Intelligence）的研究报告，将当时人工智能研究与历史上炼金术相提并论，以说明当时进行的人工智能研究是没有基础的无用之功。如果炼金术士不再关注曲颈瓶和五角器皿，而把时间花在寻找问题的深层结构，如果人从树上下来开始着手发明“火”与“车轮”，事情就会向一个更令人鼓舞的方向发展。

《大模型的安魂曲（Requiem for large-scale models）》：加州大学伯克利分校助理教授于70年代针对交通预测模型越来越复杂，撰写过《大模型的安魂曲（Requiem for large-scale models）》一文，列举了在城市交通预测领域所使用大模型的七宗罪（seven sins），认为由众多参数构成的大模型不仅不会进化、反而会崩溃（collapsed rather than evolved）。

无他、但手熟尔：众多参数构成的复杂深度模型虽然在许多任务上取得了亮丽效果，但对模型“知其所以然”的理解却举步维艰，不得不感叹复杂深度模型“无他、但手熟尔”。

本章小结

- 与手工构造特征不同，深度学习通过**“端到端(end to end)”**学习方式来形成原始数据的有效特征表达，其基本动机在于通过构建多层网络来学习隐含在数据内部的关系，从而使学习得到的特征具有更强表达力和泛化能力。
- 将深度学习应用于语义概念识别和理解时，一般思路是将深度学习得到的特征(如卷积神经网络中最后一个全连接层输出以及词向量模型中输入层与隐藏层的连接权重)输入给分类模型（如softmax函数），然后根据损失函数所计算所得“误差”来对模型进行优化。这一犹如**“黑盒子”的学习机制**使得深度学习结果缺乏解释性，从而难以可信（如易受到对抗样本攻击等），如何将知识引入学习过程，得到**可信安全、解释性更强、较少依赖标注大数据、自适应更好的深度学习**方法是目前的研究热点。

谢谢观看